

tts_transformer_tutorial

June 20, 2026

0.1 TTS Tutorial

Prepared by: Ye Kyaw Thu, Language Understanding Lab., Myanmar

Last Updated Date: 20 June 2026

For AI Engineering (Fundamental) class students

0.2 Reference

- SLR80 Dataset: <https://www.openslr.org/80/>

```
@inproceedings{oo-et-al-2020-burmese,
  title = {{Burmese Speech Corpus, Finite-State Text Normalization and Pronunciation Grammar}},
  author = {Oo, Yin May and Wattanavekin, Theeraphol and Li, Chenfang and De Silva, Pasindu},
  booktitle = {Proceedings of The 12th Language Resources and Evaluation Conference (LREC)},
  month = may,
  year = {2020},
  pages = "6328--6339",
  address = {Marseille, France},
  publisher = {European Language Resources Association (ELRA)},
  url = {https://www.aclweb.org/anthology/2020.lrec-1.777},
  ISBN = {979-10-95546-34-4},
}
```

0.3 Current Version

```
[46]: from IPython.display import Code
```

```
# Pass the path to your python file
Code(filename="myanmar_tts_transformer.py", language="python")
```

```
[46]: #!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Myanmar TTS Demo v5 - TransformerTTS + Guided Attention (No MFA)
=====
Filename: myanmar_tts_demo.py
```

WHY v1-v3 ALWAYS PRODUCED WIND (structural root cause)

All v1-v3 versions used a non-autoregressive architecture:

Text → Encoder → Gaussian Upsampling → Decoder → Mel

The Gaussian upsampling gives each mel frame a SOFT MIXTURE of character embeddings. The same mixture can correspond to many different phonemes. Under L1 loss the model predicts the CONDITIONAL MEAN of all possible targets.

*Mean of [vowel 'a' spectrum, consonant 'k' spectrum, ...]
= flat, featureless average = white noise = wind sound.*

Evidence: v3.1 dev loss plateau at 1.06 from epoch 21 → 118 (97 more epochs, only 0.024 improvement). That plateau IS the model converging to the conditional-mean prediction, not a bug.

FIX (v4): switched to an autoregressive decoder (TransformerTTS). The decoder receives the PREVIOUS MEL FRAME as an additional input, removing the averaging ambiguity entirely.

WHY v4 PRODUCED ONLY ~11 FRAMES REGARDLESS OF TEXT LENGTH

v4 trained beautifully (dev 0.97±0.25 over 300 epochs, no plateau - proof the decoder CAN model real mel spectra). But at inference, EVERY input - a 9-char word and a 62-char sentence alike - produced exactly ~11 frames of near-silent output ("kyit").

Root cause: EXPOSURE BIAS + UNCONSTRAINED CROSS-ATTENTION.

Training (teacher forcing): decoder input at step t = GT mel[$t-1$], always perfect. Cross-attention never has to recover from a bad self-generated frame.

Inference (autoregressive): decoder input at step t = its OWN prediction. Small errors compound every step. Within ~10 frames the input drifts out of the training distribution. Vanilla multi-head attention - never forced to be monotonic, and never needed to be during teacher forcing - locks onto an arbitrary text position (often near the end token) and the decoder reads "end of utterance" context → stop fires immediately.

Proof: output length was IDENTICAL regardless of input length, meaning the decoder wasn't actually reading the text encoder.

FIX (v5) - Guided Attention Loss (DC-TTS, Tachibana et al. 2017): Adds a loss term that penalises attention mass falling far from the diagonal (mel_position/mel_len text_position/text_len). This gives an explicit gradient toward the only sensible alignment for speech - monotonic, left-to-right, one pass over the text -

which is exactly the standard remedy for this failure mode on small TTS datasets (~2000 utterances, as here).

FIX (v5) - Length-aware inference floor (safety net):

min_frames/max_frames are computed from input text length $\times$ mean frames/char (from *dataset_stats.json*), so the stop-token cannot terminate generation after only ~ 11 frames regardless of how long the input text is.

Architecture (unchanged from v4):

```

Text → CharEmbed+SinPE → TransformerEncoder(N)
                                ↓ cross-attention (now guided)
[GT mel t-1] → PreNet → +SinPE → TransformerDecoder(N)
                                ↓
                                Linear → mel_before
                                Postnet → mel_after
                                Linear → stop_logits

```

Training: teacher forcing + guided attention loss on every layer

Inference: autoregressive loop with length-aware min/max frames

Expected behaviour after retraining with v5:

- Output length scales with input text length (9 chars $\rightarrow$ ~60-70 frames, 62 chars $\rightarrow$ ~400-450 frames), not a fixed ~11 frames.
- mel_db max should reach -5 to -15 dB (not -47 to -48 dB).
- Cross-attention maps (if visualised) show a clear diagonal band.

NOTE: --stage prep does NOT need to be re-run.

You MUST retrain from scratch (rm best_model.pt / checkpoints) since the architecture's loss function changed - a v4 checkpoint was never trained with guided attention and will still collapse.

UPDATED FOR AIEF CLASS: Ye Kyaw Thu, LU Lab., Myanmar

DATE: 19 June 2026

/// /// ///

```
from __future__ import annotations
```

```
import argparse, json, logging, math, os, random, sys, time, unicodedata
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional, Tuple
```

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
```

```

#
# S1 Logging & dependency check
#

def setup_logging(log_file=None, verbose=False):
    level = logging.DEBUG if verbose else logging.INFO
    fmt = "%(asctime)s [% (levelname)-8s] %(message)s"
    h = [logging.StreamHandler(sys.stdout)]
    if log_file:
        Path(log_file).parent.mkdir(parents=True, exist_ok=True)
        h.append(logging.FileHandler(log_file, encoding="utf-8"))
    logging.basicConfig(level=level, format=fmt,
                        datefmt="%Y-%m-%d %H:%M:%S", handlers=h)
    return logging.getLogger("tts")

def check_dependencies(logger):
    ok = True
    for pkg, hint in [("torch", "pip install torch"),
                     ("soundfile", "pip install soundfile"),
                     ("librosa", "pip install librosa"),
                     ("matplotlib", "pip install matplotlib"),
                     ("scipy", "pip install scipy")]:
        try:
            __import__(pkg); logger.info(f" {pkg}")
        except ImportError: logger.error(f" {pkg} → {hint}"); ok = False
    for pkg, hint in [("gi", "apt install python3-gi python3-gi-cairo gir1.
↳2-pango-1.0")]:
        try:
            __import__(pkg); logger.info(f" {pkg}")
        except ImportError: logger.info(f" - {pkg} (optional) → {hint}")
    return ok

#
# S2 Myanmar text
#

_MYA_LO, _MYA_HI = 0x1000, 0x109F

def is_myanmar_char(c): return _MYA_LO <= ord(c) <= _MYA_HI

def normalize_text(text):
    text = unicodedata.normalize("NFC", text.strip())
    return "".join(c for c in text
                   if is_myanmar_char(c) or c.isascii() or c == " ")

```

```

class MyanmarCharVocab:
    PAD, UNK, BOS, EOS = "<pad>", "<unk>", "<bos>", "<eos>"

    def __init__(self):
        self.char2idx = {self.PAD:0, self.UNK:1, self.BOS:2, self.EOS:3}
        self.idx2char = {v:k for k,v in self.char2idx.items()}

    def build(self, texts):
        for t in texts:
            for c in normalize_text(t):
                if c not in self.char2idx:
                    i = len(self.char2idx)
                    self.char2idx[c] = i; self.idx2char[i] = c

    def encode(self, text):
        n = normalize_text(text)
        return ([self.char2idx[self.BOS]]
                + [self.char2idx.get(c, self.char2idx[self.UNK]) for c in n]
                + [self.char2idx[self.EOS]])

    def decode(self, ids):
        sp = {self.PAD, self.UNK, self.BOS, self.EOS}
        return "".join(self.idx2char.get(i, self.UNK)
                       for i in ids if self.idx2char.get(i, self.UNK) not in sp)

    @property
    def size(self): return len(self.char2idx)

    def save(self, path):
        with open(path, "w", encoding="utf-8") as f:
            json.dump({"char2idx": self.char2idx}, f, ensure_ascii=False)

    @classmethod
    def load(cls, path):
        with open(path, encoding="utf-8") as f: d = json.load(f)
        v = cls()
        v.char2idx = {k: int(i) for k,i in d["char2idx"].items()}
        v.idx2char = {int(i): k for k,i in d["char2idx"].items()}
        return v

#
# §3 Mel utilities - fixed [-80,0]+[-1,+1] normalisation
#

_MEL_MIN_DB, _MEL_MAX_DB, _MEL_RANGE = -80.0, 0.0, 80.0

def mel_normalize(mel_db):

```

```

    return (2.0*(np.clip(mel_db, _MEL_MIN_DB, _MEL_MAX_DB)-_MEL_MIN_DB)
            /_MEL_RANGE - 1.0).astype(np.float32)

def mel_denormalize(mel_norm):
    return ((np.clip(mel_norm, -1.0, 1.0)+1.0)/2.0
            *_MEL_RANGE+_MEL_MIN_DB).astype(np.float32)

@dataclass
class MelConfig:
    sample_rate: int = 22050
    n_fft: int = 1024
    hop_length: int = 256
    win_length: int = 1024
    n_mels: int = 80
    fmin: float = 0.0
    fmax: float = 8000.0

    def to_dict(self): return {k:v for k,v in self.__dict__.items()}

    @classmethod
    def from_dict(cls, d):
        return cls(**{k:v for k,v in d.items()
                       if k in cls.__dataclass_fields__})

def load_audio(path, sr):
    import soundfile as sf
    a, s = sf.read(path, dtype="float32")
    if a.ndim > 1: a = a.mean(1)
    if s != sr:
        import librosa; a = librosa.resample(a, orig_sr=s, target_sr=sr)
    return a

def audio_to_mel(audio, cfg):
    import librosa
    m = librosa.feature.melspectrogram(
        y=audio, sr=cfg.sample_rate, n_fft=cfg.n_fft,
        hop_length=cfg.hop_length, win_length=cfg.win_length,
        n_mels=cfg.n_mels, fmin=cfg.fmin, fmax=cfg.fmax)
    return mel_normalize(librosa.power_to_db(m, ref=np.max).T) # (T,n_mels)

def mel_to_audio_gl(mel_norm, cfg, n_iter=200):
    """Normalized mel → audio via improved Griffin-Lim (200 iter, power=1.5)."""
    import librosa
    from scipy.signal import butter, sosfiltfilt

```

```

p = librosa.db_to_power(mel_denormalize(mel_norm).T.astype(np.float64))
a = librosa.feature.inverse.mel_to_audio(
    p, sr=cfg.sample_rate, n_fft=cfg.n_fft,
    hop_length=cfg.hop_length, win_length=cfg.win_length,
    n_iter=n_iter, power=1.5, center=True)
nyq = cfg.sample_rate/2.0
sos = butter(4, [max(80.,cfg.fmin+20)/nyq,
                 min(7800.,cfg.fmax-200)/nyq],
             btype="band", output="sos")
a = sosfiltfilt(sos, a)
pk = np.max(np.abs(a))
if pk > 1e-8: a = a/pk*0.9
return a.astype(np.float32)

def save_wav(audio, path, sr):
    import soundfile as sf; sf.write(path, audio.astype(np.float32), sr)

#
# S4 Vocoder wrapper
#

class Vocoder:
    def __init__(self, cfg, logger, use_neural=True):
        self.cfg = cfg; self.logger = logger; self._wg = None
        if use_neural:
            try:
                wg = torch.hub.load("NVIDIA/DeepLearningExamples:torchhub",
                                    "nvidia_waverglow", model_math="fp32",
                                    pretrained=True, verbose=False)
                wg.eval()
                for m in wg.modules():
                    if hasattr(m, "weight_g"):
                        try: nn.utils.remove_weight_norm(m)
                        except: pass
                self._wg = wg; logger.info("WaveGlow loaded ")
            except Exception as e:
                logger.warning(f"WaveGlow unavailable ({type(e).__name__}), "
                               "using Griffin-Lim.")

    def synthesize(self, mel_norm):
        if self._wg is not None: return self._waverglow(mel_norm)
        return mel_to_audio_gl(mel_norm, self.cfg)

    def _waverglow(self, mel_norm):
        t = torch.from_numpy(mel_denormalize(mel_norm).T).float().unsqueeze(0)
        d = next(self._wg.parameters()).device

```

```

        with torch.no_grad(): a = self._wg.infer(t.to(d), sigma=0.9)
        a = a.squeeze().cpu().numpy()
        pk = np.max(np.abs(a)); return (a/pk*0.9).astype(np.float32) if pk>1e-8
    else a

#
# §5 Shared encoder building blocks
#

class SinusoidalPE(nn.Module):
    def __init__(self, d_model, max_len=5000, dropout=0.1):
        super().__init__()
        self.drop = nn.Dropout(dropout)
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(max_len, dtype=torch.float32).unsqueeze(1)
        div = torch.exp(torch.arange(0, d_model, 2, dtype=torch.float32)
                        * (-math.log(10000.0)/d_model))
        pe[:,0::2] = torch.sin(pos*div); pe[:,1::2] = torch.cos(pos*div)
        self.register_buffer("pe", pe.unsqueeze(0))

    def forward(self, x):
        return self.drop(x + self.pe[:, :x.size(1)])

class ConvFFN(nn.Module):
    def __init__(self, d_model, d_ff, kernel_size=9, dropout=0.1):
        super().__init__()
        pad = (kernel_size-1)//2
        self.c1 = nn.Conv1d(d_model, d_ff, kernel_size, padding=pad)
        self.c2 = nn.Conv1d(d_ff, d_model, kernel_size, padding=pad)
        self.norm = nn.LayerNorm(d_model)
        self.drop = nn.Dropout(dropout)

    def forward(self, x):
        r = x; x = x.transpose(1,2)
        x = self.drop(F.relu(self.c1(x)))
        return self.norm(r + self.drop(self.c2(x).transpose(1,2)))

class TransformerEncoderBlock(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, dropout=0.1, ffn_kernel=9):
        super().__init__()
        self.attn = nn.MultiheadAttention(d_model, n_heads,
                                          dropout=dropout, batch_first=True)
        self.ffn = ConvFFN(d_model, d_ff, ffn_kernel, dropout)
        self.norm1 = nn.LayerNorm(d_model)

```



```

        self.drop = nn.Dropout(dropout)

    def forward(self, x, key_padding_mask=None):
        a, _ = self.attn(x, x, x, key_padding_mask=key_padding_mask)
        x = self.norm1(x + self.drop(a))
        return self.ffn(x)

#
# S6 TransformerTTS - autoregressive decoder
#

class PreNet(nn.Module):
    """
    Two-layer MLP pre-net for mel decoder input.

    CRITICAL: dropout kept ACTIVE at inference (training=True).
    Tacotron2 paper finding: always-on dropout prevents the decoder
    from memorising training mel sequences and forces it to rely on
    the encoder text context. Without this, inference collapses.
    """
    def __init__(self, n_mels, d_model, dropout=0.5):
        super().__init__()
        self.fc1 = nn.Linear(n_mels, d_model)
        self.fc2 = nn.Linear(d_model, d_model)
        self.p = dropout

    def forward(self, x):
        # training=True → dropout always active (also at eval/inference)
        x = F.dropout(F.relu(self.fc1(x)), p=self.p, training=True)
        x = F.dropout(F.relu(self.fc2(x)), p=self.p, training=True)
        return x

class TransformerDecoderBlock(nn.Module):
    """
    Transformer decoder layer:
    1. Causal (masked) self-attention on mel frames
    2. Cross-attention to encoder output (learns text-mel alignment)
    3. Conv FFN
    """
    def __init__(self, d_model, n_heads, d_ff, dropout=0.1, ffn_kernel=9):
        super().__init__()
        self.self_attn = nn.MultiheadAttention(d_model, n_heads,
                                                dropout=dropout,
batch_first=True)
        self.cross_attn = nn.MultiheadAttention(d_model, n_heads,

```

```

dropout=dropout,
batch_first=True)
self.ffn = ConvFFN(d_model, d_ff, ffn_kernel, dropout)
self.norm1 = nn.LayerNorm(d_model)
self.norm2 = nn.LayerNorm(d_model)
self.drop = nn.Dropout(dropout)

def forward(self, x, enc_out,
            self_mask=None, enc_key_padding_mask=None,
            need_weights=False):
    # 1. Causal self-attention
    sa, _ = self.self_attn(x, x, x, attn_mask=self_mask, need_weights=False)
    x = self.norm1(x + self.drop(sa))
    # 2. Cross-attention to encoder
    # need_weights=True (training only) returns (B, T_mel, T_text)
    # attention map, averaged over heads - used for guided-attention loss.
    ca, attn_w = self.cross_attn(
        x, enc_out, enc_out,
        key_padding_mask=enc_key_padding_mask,
        need_weights=need_weights, average_attn_weights=True)
    x = self.norm2(x + self.drop(ca))
    # 3. Conv FFN
    return self.ffn(x), attn_w

class Postnet(nn.Module):
    """5-layer Conv1d postnet (explicit layers)."""
    def __init__(self, n_mels, d_hidden=512, dropout=0.1):
        super().__init__()
        self.c0 = nn.Conv1d(n_mels, d_hidden, 5, padding=2)
        self.b0 = nn.BatchNorm1d(d_hidden)
        self.c1 = nn.Conv1d(d_hidden, d_hidden, 5, padding=2)
        self.b1 = nn.BatchNorm1d(d_hidden)
        self.c2 = nn.Conv1d(d_hidden, d_hidden, 5, padding=2)
        self.b2 = nn.BatchNorm1d(d_hidden)
        self.c3 = nn.Conv1d(d_hidden, d_hidden, 5, padding=2)
        self.b3 = nn.BatchNorm1d(d_hidden)
        self.c4 = nn.Conv1d(d_hidden, n_mels, 5, padding=2)
        self.b4 = nn.BatchNorm1d(n_mels)
        self.drop = nn.Dropout(dropout)

    def forward(self, x):
        # x: (B,T,n_mels)
        x = x.transpose(1,2)
        x = self.drop(torch.tanh(self.b0(self.c0(x))))
        x = self.drop(torch.tanh(self.b1(self.c1(x))))
        x = self.drop(torch.tanh(self.b2(self.c2(x))))
        x = self.drop(torch.tanh(self.b3(self.c3(x))))

```

```

x = self.drop(self.b4(self.c4(x)))
return x.transpose(1,2)          # (B,T,n_mels)

class TransformerTTS(nn.Module):
    """
    Autoregressive Transformer TTS.

    Training (teacher forcing, fully parallel):
    model(text_ids, mel_targets, mel_lengths)
    → (mel_before, mel_after, stop_logit, loss)

    Inference (autoregressive, frame-by-frame):
    mel_norm = model.infer(text_ids) # returns (T, n_mels) numpy array
    """

    def __init__(self, vocab_size, n_mels,
                  d_model=256, n_heads=4,
                  n_enc_layers=4, n_dec_layers=4,
                  d_ff=1024, ffn_kernel=9, dropout=0.1):
        super().__init__()
        self.n_mels = n_mels

        # Encoder
        self.embed = nn.Embedding(vocab_size, d_model, padding_idx=0)
        self.enc_pe = SinusoidalPE(d_model, dropout=dropout)
        self.encoder = nn.ModuleList([
            TransformerEncoderBlock(d_model, n_heads, d_ff, dropout, ffn_kernel)
            for _ in range(n_enc_layers)])

        # Decoder
        self.prenet = PreNet(n_mels, d_model, dropout=0.5)
        self.dec_pe = SinusoidalPE(d_model, dropout=dropout)
        self.decoder = nn.ModuleList([
            TransformerDecoderBlock(d_model, n_heads, d_ff, dropout, ffn_kernel)
            for _ in range(n_dec_layers)])

        # Output heads
        self.mel_proj = nn.Linear(d_model, n_mels)
        self.stop_proj = nn.Linear(d_model, 1)
        self.postnet = Postnet(n_mels, min(d_model*2, 512), dropout)

        # private

    def _encode(self, text_ids):
        pad_mask = (text_ids == 0)          # (B, T_text)
        x = self.enc_pe(self.embed(text_ids))
        for blk in self.encoder:

```

```

        x = blk(x, key_padding_mask=pad_mask)
    return x, pad_mask                                # enc_out, enc_pad_mask

def _causal_mask(self, T, device):
    """Upper-triangular bool mask: position i cannot attend to j > i."""
    return torch.triu(
        torch.ones(T, T, device=device, dtype=torch.bool), diagonal=1)

# forward (training - teacher forcing)

def forward(self, text_ids, mel_targets, mel_lengths):
    """
    Teacher-forced training forward.

    mel_targets : (B, T_mel, n_mels)  GT mel, normalized to [-1,+1]
    mel_lengths : (B,)                real frame counts (for masking)

    At each position t the decoder input is GT mel[t-1] (shifted right).
    The entire sequence is processed IN PARALLEL with a causal mask -
    training is as fast as a non-autoregressive model.
    """

    B, T_mel, _ = mel_targets.shape
    dev = text_ids.device

    enc_out, enc_pad = self._encode(text_ids)
    text_lengths = (~enc_pad).sum(dim=1)                # (B,) real chars

    # Shift mel right: prepend silence (zeros), drop last frame
    zeros = torch.zeros(B, 1, self.n_mels, device=dev)
    dec_in = torch.cat([zeros,
                        mel_targets[:, :-1].clamp(-1.0, 1.0)], dim=1)

    dec_h = self.dec_pe(self.prenet(dec_in))            # (B,T_mel,d)
    causal = self._causal_mask(T_mel, dev)

    # Collect cross-attention maps from every layer for guided-attention
    # loss (FIX: forces monotonic left-to-right alignment - see
    # guided_attention_loss() docstring for why this is necessary).
    attn_maps = []
    for layer in self.decoder:
        dec_h, attn_w = layer(dec_h, enc_out,
                              self_mask=causal,
                              enc_key_padding_mask=enc_pad,
                              need_weights=True)
        attn_maps.append(attn_w)                        # (B,T_mel,T_text)

    mel_before = self.mel_proj(dec_h)                  # (B,T_mel,n_mels)
    mel_after = mel_before + self.postnet(mel_before)

```

```

stop_logit = self.stop_proj(dec_h).squeeze(-1)          # (B,T_mel)

# Mel loss (masked: only real frames)
mel_loss = masked_mel_loss(mel_before, mel_after, mel_targets,
                             mel_lengths)

# Guided attention loss (averaged over all decoder layers)
# THE FIX for the 11-frame / wind collapse: without this, cross-
# attention is free to ignore the text and the decoder finds the
# shortcut of stopping almost immediately. This loss forces a
# monotonic diagonal alignment, which is the only correct alignment
# for speech (text is read left-to-right, once, no skipping).
ga_loss = torch.stack([
    guided_attention_loss(a, text_lengths, mel_lengths)
    for a in attn_maps
]).mean()

# Stop token loss
# Target: 1.0 at the last frame of each utterance
stop_tgt = torch.zeros(B, T_mel, device=dev)
for b in range(B):
    last = min(int(mel_lengths[b].item())-1, T_mel-1)
    stop_tgt[b, last] = 1.0

frame_idx = torch.arange(T_mel, device=dev).unsqueeze(0)
stop_mask = (frame_idx < mel_lengths.to(dev).unsqueeze(1)).float()

# positive weight = total frames / 1 positive frame (balanced BCE)
pos_w = stop_mask.sum(dim=1).clamp(min=2.0)          # (B,)
stop_loss = torch.stack([
    F.binary_cross_entropy_with_logits(
        stop_logit[b], stop_tgt[b],
        pos_weight=pos_w[b:b+1],
        reduction="none"
    ).mul(stop_mask[b]).sum() / stop_mask[b].sum().clamp(min=1)
    for b in range(B)
]).mean()

total = mel_loss + 0.1 * stop_loss + 1.0 * ga_loss
return mel_before, mel_after, stop_logit, total

# infer (autoregressive)

def infer(self, text_ids, max_frames=2000, stop_thresh=0.5, min_frames=10):
    """
    Autoregressive inference: generates mel one frame at a time.

```

Why it works (and the non-AR approach did not):

At step t , the decoder receives the PREVIOUS predicted mel frame. This provides the crucial disambiguating signal: given what was just spoken, the model predicts what comes next. No ambiguity, no averaging, no wind.

min_frames: hard floor on output length, computed by the caller from text length $\times$ expected frames/char (see synthesizer_text). SAFETY NET: even if attention alignment is imperfect, this stops the stop-token from firing within the first few frames - the exact failure mode that produced 11-frame outputs regardless of input text length. The real fix is guided_attention_loss() during training; this floor is a robustness backstop.

Returns: (T, n_mels) float32 numpy array, normalized mel [-1,+1]
"""

```
enc_out, enc_pad = self._encode(text_ids)
dev = text_ids.device

frames: List[torch.Tensor] = []
# Start with one silence frame (zeros)
dec_in = torch.zeros(1, 1, self.n_mels, device=dev)

for t in range(max_frames):
    T = dec_in.size(1)
    dec_h = self.dec_pe(self.prenet(dec_in))
    causal = self._causal_mask(T, dev)

    for layer in self.decoder:
        dec_h, _ = layer(dec_h, enc_out,
                        self_mask=causal,
                        enc_key_padding_mask=enc_pad,
                        need_weights=False)

    last_h = dec_h[:, -1:] # (1,1,d)
    mel_frame = self.mel_proj(last_h) # (1,1,n_mels)
    stop_prob = torch.sigmoid(self.stop_proj(last_h)) # (1,1,1)

    frames.append(mel_frame[0, 0]) # (n_mels,)
    dec_in = torch.cat([dec_in, mel_frame], dim=1) # grow by 1

    if stop_prob.item() > stop_thresh and t >= min_frames:
        break

if not frames:
    return np.zeros((1, self.n_mels), dtype=np.float32)

mel_pred = torch.stack(frames).unsqueeze(0) # (1,T,n_mels)
```

```

        mel_after = mel_pred + self.postnet(mel_pred)
        return mel_after[0].detach().cpu().numpy() # (T,n_mels)

#
# §7 Loss
#

def make_length_mask(lengths, max_len):
    """(B, max_len, 1) float mask: 1.0 for real frames, 0.0 for padding."""
    idx = torch.arange(max_len, device=lengths.device).unsqueeze(0)
    return (idx < lengths.unsqueeze(1)).unsqueeze(-1).float()

def guided_attention_loss(attn_w, text_lengths, mel_lengths, g=0.2):
    """
    Guided Attention Loss (DC-TTS, Tachibana et al. 2017).

    WHY THIS IS NEEDED (root cause of the "11-frame / wind" failure)

    Vanilla cross-attention is free to attend ANYWHERE in the text at
    each decoder step. With only ~2000 training utterances it never
    discovers on its own that attention should sweep monotonically
    left-to-right as time progresses. Instead it found a shortcut:
    mostly ignore the text, predict "stop" almost immediately.
    Evidence: a 9-char and a 62-char input both produced exactly the
    same ~11 frames - proof the decoder wasn't actually reading the
    encoder output.

    This loss penalises attention mass that falls far from the
    diagonal line (text_position/text_len == mel_position/mel_len).
    It gives the model an explicit, strong gradient toward the only
    sensible alignment for speech: monotonic, left-to-right, one
    pass over the text. This is the standard fix in TTS literature
    for exactly this failure mode on small datasets.

    attn_w      : (B, T_mel, T_text) - averaged-over-heads attention
                  weights from one decoder layer (rows sum to 1)
    text_lengths : (B,) real (non-padded) text token counts
    mel_lengths  : (B,) real (non-padded) mel frame counts
    g            : float - band width; smaller = stricter diagonal
    """
    B, T_mel, T_text = attn_w.shape
    device = attn_w.device
    losses = []
    for b in range(B):
        Tm = min(int(mel_lengths[b].item()), T_mel)

```

```

    Tt = min(int(text_lengths[b].item()), T_text)
    if Tm < 1 or Tt < 1:
        continue
    n = torch.arange(Tm, device=device, dtype=torch.float32).unsqueeze(1) /
↳max(Tm-1, 1)
    t = torch.arange(Tt, device=device, dtype=torch.float32).unsqueeze(0) /
↳max(Tt-1, 1)
    W = 1.0 - torch.exp(-((n - t) ** 2) / (2.0 * g * g))    # (Tm, Tt), 0 on
↳diagonal
    losses.append((attn_w[b, :Tm, :Tt] * W).mean())
    if not losses:
        return torch.zeros((), device=device)
    return torch.stack(losses).mean()

def masked_mel_loss(mel_before, mel_after, targets, mel_lengths):
    """
    Masked L1 + Masked Spectral Convergence.

    CRITICAL: divides by n_valid_ELEMENTS (frames × mel_bins), not just frames.
    Previous bug: mask.sum() counted frames only → loss 80× too large →
    effective lr = 1e-3/80 = 1.25e-5 → model barely trained.
    """
    tgt = targets.clamp(-1.0, 1.0)
    B, T, M = mel_before.shape

    if mel_lengths is not None:
        mask = make_length_mask(mel_lengths, T).to(mel_before.device)
        n_el = (mask.sum() * M).clamp(min=1.0)    # frames × bins

        l1_b = ((mel_before - tgt).abs() * mask).sum() / n_el
        l1_a = ((mel_after - tgt).abs() * mask).sum() / n_el

        pm = mel_after * mask; tm = tgt * mask
        sc = ((tm-pm).norm(p="fro",dim=(-2,-1)) /
              tm.norm(p="fro",dim=(-2,-1)).clamp(min=1e-8)).mean()
    else:
        l1_b = F.l1_loss(mel_before, tgt)
        l1_a = F.l1_loss(mel_after, tgt)
        diff = (tgt - mel_after).norm(p="fro", dim=(-2,-1))
        sc = (diff / tgt.norm(p="fro",dim=(-2,-1)).clamp(min=1e-8)).mean()

    return l1_b + l1_a + sc

#
# §8 Dataset & DataLoader (3-tuple: text_ids, mel_norm, mel_length)

```



```

#

class TTSDataset(Dataset):
    def __init__(self, samples): self.samples = samples
    def __len__(self): return len(self.samples)

    def __getitem__(self, idx):
        s = self.samples[idx]
        ti = torch.tensor(s["text_ids"], dtype=torch.long)
        m = torch.from_numpy(np.load(s["mel"]).astype(np.float32))
        ml = torch.tensor(m.shape[0], dtype=torch.long)
        return ti, m, ml

def _collate(batch):
    texts, mels, mlens = zip(*batch)
    B = len(texts)
    max_t = max(t.size(0) for t in texts)
    max_m = max(m.size(0) for m in mels)
    n_mels = mels[0].size(1)

    tp = torch.zeros(B, max_t, dtype=torch.long)
    mp = torch.full((B, max_m, n_mels), -2.0) # -2.0 sentinel (outside [-1,+1])
    for i,(t,m) in enumerate(zip(texts,mels)):
        tp[i,:t.size(0)] = t
        mp[i,:m.size(0)] = m
    return tp, mp, torch.stack(list(mlens))

def make_dataloader(samples, batch_size, shuffle=True, num_workers=2):
    ds = TTSDataset(samples)
    return DataLoader(ds, batch_size=batch_size, shuffle=shuffle,
                      num_workers=num_workers, collate_fn=_collate,
                      pin_memory=torch.cuda.is_available(),
                      drop_last=(shuffle and len(ds) >= batch_size*2))

#
# §9 LR scheduler (linear warmup + cosine decay)
#

def warmup_cosine(optimizer, warmup_epochs, total_epochs, min_lr=0.05):
    def _lr(ep):
        if ep < warmup_epochs: return (ep+1)/max(warmup_epochs,1)
        p = (ep-warmup_epochs)/max(total_epochs-warmup_epochs,1)
        return min_lr + (1-min_lr)*0.5*(1+math.cos(math.pi*p))
    return optim.lr_scheduler.LambdaLR(optimizer, _lr)

```

```

#
# §10 Mel plots
#

def _pango_render(text, font_path, size=14):
    try:
        import gi; gi.require_version("Pango","1.0"); gi.
        require_version("PangoCairo","1.0")
        from gi.repository import Pango, PangoCairo; import cairo
        t = cairo.ImageSurface(cairo.FORMAT_ARGB32,1,1); tc=cairo.Context(t)
        lay=PangoCairo.create_layout(tc); fd=Pango.FontDescription.
        from_string(f"Myanmar3 {size}")
        lay.set_font_description(fd); lay.set_text(text,-1)
        w,h=lay.get_pixel_size(); p=6; w+=2*p; h+=2*p
        s=cairo.ImageSurface(cairo.FORMAT_ARGB32,w,h); c=cairo.Context(s)
        c.set_source_rgb(0,0,0); l=PangoCairo.create_layout(c)
        l.set_font_description(fd); l.set_text(text,-1)
        c.move_to(p,p); PangoCairo.show_layout(c,l)
        r=np.frombuffer(s.get_data(),dtype=np.uint8).reshape(h,w,4)
        return r[...,[2,1,0,3]].copy()
    except: return None

def plot_mel(mel, title, save_path, logger, vmin=-1.0, vmax=1.0):
    try:
        import matplotlib; matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        from matplotlib import font_manager
        from matplotlib.offsetbox import AnnotationBbox, OffsetImage
        mf="/usr/share/fonts/truetype/mm3-multi-os.ttf"
        if os.path.exists(mf):
            font_manager.fontManager.addfont(mf)
            fp=font_manager.FontProperties(fname=mf)
            plt.rcParams["font.family"]="sans-serif"
            plt.rcParams["font.sans-serif"]=[fp.get_name(),"DejaVu Sans"]
        fig,ax=plt.subplots(figsize=(12,4))
        im=ax.imshow(mel.T,aspect="auto",origin="lower",cmap="viridis",
                     vmin=vmin,vmax=vmax,interpolation="none")
        plt.colorbar(im,ax=ax,label="Normalised mel [-1,+1]")
        ax.set_xlabel("Frame"); ax.set_ylabel("Mel bin")
        rgba=_pango_render(title,mf)
        if rgba is not None:
            ab=AnnotationBbox(OffsetImage(rgba,zoom=1),(0.5,1.12),
                             xycoords="axes fraction",
                             box_alignment=(0.5,0.5),frameon=False)
            ax.add_artist(ab)

```

```

        else:
            ax.set_title(title.encode("ascii","replace").decode()[:
↵80],fontsize=9)
            plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
            plt.close(fig); logger.info(f"Mel plot → {save_path}")
        except Exception as e: logger.warning(f"Mel plot failed: {e}")

def plot_comparison(pred, gt, save_path, logger):
    try:
        import matplotlib; matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        fig,axes=plt.subplots(2,1,figsize=(12,6))
        for ax,m,t in zip(axes,[gt,pred],["GT mel","TTS predicted mel"]):
            im=ax.imshow(m.T,aspect="auto",origin="lower",cmap="viridis",
                           vmin=-1,vmax=1,interpolation="none")
            ax.set_title(t); ax.set_xlabel("Frame"); ax.set_ylabel("Mel bin")
            plt.colorbar(im,ax=ax)
        plt.tight_layout(); plt.savefig(save_path,dpi=150); plt.close(fig)
        logger.info(f"Comparison plot → {save_path}")
    except Exception as e: logger.warning(f"Comparison plot failed: {e}")

#
# §11 Data preparation
#

def _load_split(output_dir, split):
    p = Path(output_dir)/f"{split}.json"
    if not p.exists(): return []
    with open(p, encoding="utf-8") as f: return json.load(f)

def prepare_data(data_dir, tsv_file, output_dir, train_ratio, dev_ratio,
                 seed, mel_cfg, max_samples, logger):
    random.seed(seed); np.random.seed(seed)
    dp = Path(data_dir); tp = dp/tsv_file
    if not tp.exists(): logger.error(f"TSV not found: {tp}"); sys.exit(1)
    wd = dp/"wavs"

    def _wav(fid):
        for p in [wd/f"{fid}.wav", dp/f"{fid}.wav"]:
            if p.exists(): return p

    raw = []
    with open(tp, encoding="utf-8") as f:
        for line in f:

```

```

        ps = line.strip().split(maxsplit=1)
        if len(ps)!=2: continue
        fid,text = ps; w=_wav(fid)
        if w: raw.append({"id":fid,"wav":str(w),"text":text})

if max_samples>0 and len(raw)>max_samples:
    random.shuffle(raw); raw=raw[:max_samples]
logger.info(f"Found {len(raw)} audio files.")

vocab = MyanmarCharVocab(); vocab.build([s["text"] for s in raw])
logger.info(f"Vocabulary: {vocab.size} characters")

out = Path(output_dir); out.mkdir(parents=True,exist_ok=True)
(out/"mels").mkdir(exist_ok=True)
vocab.save(str(out/"vocab.json"))
with open(out/"mel_config.json","w") as f: json.dump(mel_cfg.to_dict(),f)
with open(out/"normalization.json","w") as f:
    json.dump({"type":"fixed_linear","mel_min_db":_MEL_MIN_DB,
               "mel_max_db":_MEL_MAX_DB,"norm_min":-1.0,"norm_max":1.0,
               "formula":"norm=2*(db-(-80))/80-1"},f)

logger.info("Computing mel spectrograms ...")
valid=[]; skip=0
for i,s in enumerate(raw):
    try:
        a=load_audio(s["wav"],mel_cfg.sample_rate)
        d=len(a)/mel_cfg.sample_rate
        if d<0.3 or d>15.0: skip+=1; continue
        mn=audio_to_mel(a,mel_cfg)
        mp=out/"mels"/f"{s['id']}.npz"; np.save(mp,mn)
        ti=vocab.encode(s["text"])
        valid.append({"id":s["id"],"wav":s["wav"],"text":s["text"],
                      "text_ids":ti,"mel":str(mp),
                      "mel_frames":int(mn.shape[0]),"text_len":len(ti)})
    except Exception as e: logger.debug(f"Skip {s['id']}: {e}"); skip+=1
    if (i+1)%500==0: logger.info(f" {i+1}/{len(raw)} ...")

logger.info(f"Valid: {len(valid)} | Skipped: {skip}")

ml=[s["mel_frames"] for s in valid]; tl=[s["text_len"] for s in valid]
fpc=[m/t for m,t in zip(ml,tl)]
mfpc=float(np.mean(fpc))
smp=[np.load(s["mel"]) for s in valid[:50]]
logger.info(f"Mel norm check: min={np.min([m.min() for m in smp]):.3f} "
            f"max={np.max([m.max() for m in smp]):.3f} "
            f"mean={np.mean([m.mean() for m in smp]):.3f}")
logger.info(f"Mel frames : mean={np.mean(ml):.0f} ± {np.std(ml):.0f}")
logger.info(f"Frames/char: mean={mfpc:.1f} ± {np.std(fpc):.1f}")

```

```

with open(out/"dataset_stats.json","w") as f:
    json.dump({"mean_frames_per_char":mfpc,
              "mean_mel_frames":float(np.mean(ml)),
              "n_utterances":len(valid)},f)

random.shuffle(valid)
n=len(valid); nt=int(n*train_ratio); nd=int(n*dev_ratio)
splits={"train":valid[:nt],"dev":valid[nt:nt+nd],"test":valid[nt+nd:]}
for name,data in splits.items():
    with open(out/f"{name}.json","w",encoding="utf-8") as f:
        json.dump(data,f,ensure_ascii=False)
    logger.info(f" {name}: {len(data)}")
logger.info("Data preparation complete ")

#
# §12 Training
#

def _load_artifacts(output_dir):
    out=Path(output_dir)
    v=MyanmarCharVocab.load(str(out/"vocab.json"))
    with open(out/"mel_config.json") as f: mc=MelConfig.from_dict(json.load(f))
    return v, mc

def _load_mean_fpc(output_dir, logger=None, default=7.3):
    """
    Load mean frames-per-char from dataset_stats.json (written by prepare_data).
    Used to compute a length-aware min_frames floor at inference, so output
    duration tracks input text length instead of relying solely on a
    stop-token that can fire prematurely (see infer() docstring).
    """
    p = Path(output_dir)/"dataset_stats.json"
    if p.exists():
        try:
            with open(p) as f:
                return float(json.load(f)["mean_frames_per_char"])
        except Exception:
            pass
    if logger:
        logger.warning(f"dataset_stats.json not found; using fallback "
                      f"mean_frames_per_char={default}")
    return default

def _build_model(args, vocab_size, n_mels, device):

```

```

return TransformerTTS(
    vocab_size=vocab_size, n_mels=n_mels,
    d_model=args.d_model, n_heads=args.n_heads,
    n_enc_layers=args.n_layers, n_dec_layers=args.n_layers,
    d_ff=args.ffn_dim, dropout=args.dropout
).to(device)

def _save_ckpt(path, model, opt, sched, epoch, dev_loss, best, args):
    torch.save({"epoch":epoch,"model_state":model.state_dict(),
        "opt_state":opt.state_dict(),"sched_state":sched.state_dict(),
        "dev_loss":dev_loss,"best_dev_loss":best,
        "model_config":{
            "arch":"TransformerTTS",
            "vocab_size":model.embed.num_embeddings,
            "n_mels":model.n_mels,
            "d_model":args.d_model,"n_heads":args.n_heads,
            "n_layers":args.n_layers,"ffn_dim":args.ffn_dim,
            "dropout":args.dropout}}, path)

def run_train(args, logger):
    out = Path(args.output_dir); out.mkdir(parents=True, exist_ok=True)
    vocab, mel_cfg = _load_artifacts(args.output_dir)
    train_s = _load_split(args.output_dir,"train")
    dev_s = _load_split(args.output_dir,"dev")
    if not train_s: logger.error("No training data. Run --stage prep."); sys.
    exit(1)

    logger.info(f"Vocab:{vocab.size} Train:{len(train_s)} Dev:{len(dev_s)}")
    dev = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    logger.info(f"Device: {dev}")

    model = _build_model(args, vocab.size, mel_cfg.n_mels, dev)
    np_ = sum(p.numel() for p in model.parameters())
    logger.info(f"TransformerTTS parameters: {np_,}")

    opt = optim.AdamW(model.parameters(), lr=args.lr,
        betas=(0.9,0.98), eps=1e-9,
        weight_decay=args.weight_decay)
    sched = warmup_cosine(opt, args.warmup_epochs, args.epochs)

    start=1; best=float("inf")
    if args.resume:
        cp=Path(args.resume)
        if not cp.exists(): logger.error(f"Checkpoint not found: {cp}"); sys.
    exit(1)

```

```

ck=torch.load(cp,map_location=dev,weights_only=False)
model.load_state_dict(ck["model_state"])
if "opt_state" in ck: opt.load_state_dict(ck["opt_state"])
if "sched_state" in ck: sched.load_state_dict(ck["sched_state"])
start=ck.get("epoch",0)+1; best=ck.get("best_dev_loss",float("inf"))
logger.info(f"Resumed from epoch {start-1} (best dev={best:.4f})")

tr_dl = make_dataloader(train_s, args.batch_size,
                        shuffle=True, num_workers=args.num_workers)
dv_dl = make_dataloader(dev_s, args.batch_size,
                        shuffle=False, num_workers=args.num_workers)

logger.info(f"Training {args.epochs} epochs "
            f"(warmup={args.warmup_epochs}, lr={args.lr:.1e})")
logger.info("Architecture: TransformerTTS + Guided Attention Loss (v5)")
logger.info("Expected: ep1 0.8-1.1 ep50 0.35-0.55 ep200 0.10-0.25")
logger.info("v4 bug: stop fired ~11 frames regardless of text length␣
↳(exposure")
logger.info("bias + unconstrained attention). Guided attention loss forces")
logger.info("monotonic left-to-right alignment, fixing the early-stop␣
↳collapse.")

history = []
for epoch in range(start, args.epochs+1):

    # train
    model.train(); sl=0.0; nb=0
    for text_ids, mel_tgt, mel_lens in tr_dl:
        text_ids = text_ids.to(dev)
        mel_tgt = mel_tgt.to(dev)
        mel_lens = mel_lens.to(dev)

        _,_, loss = model(text_ids, mel_tgt, mel_lens)
        opt.zero_grad(); loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
        opt.step()
        sl += loss.item(); nb += 1

    if args.verbose and nb % args.log_steps == 0:
        logger.debug(f" Ep{epoch:3d} s{nb:4d} | "
                    f"loss={loss.item():.4f} "
                    f"lr={opt.param_groups[0]['lr']:.2e}")

    avg_tr = sl/max(nb,1); sched.step()

    # validate
    model.eval(); sd=0.0

```

```

with torch.no_grad():
    for text_ids, mel_tgt, mel_lens in dv_dl:
        _,_, loss = model(text_ids.to(dev), mel_tgt.to(dev),
                           mel_lens.to(dev))
        sd += loss.item()
dev_loss = sd/max(len(dv_dl),1)
lr = opt.param_groups[0]["lr"]

logger.info(f"Epoch {epoch:4d}/{args.epochs} | "
            f"train={avg_tr:.4f} dev={dev_loss:.4f} | lr={lr:.2e}")
history.append({"epoch":epoch,"train":avg_tr,"dev":dev_loss,"lr":lr})

if dev_loss < best:
    best = dev_loss
    _save_ckpt(out/"best_model.pt", model, opt, sched,
               epoch, dev_loss, best, args)
    logger.info(f"    Best model saved (dev={dev_loss:.4f})")

if epoch % args.save_every == 0:
    _save_ckpt(out/f"checkpoint_ep{epoch:04d}.pt",
               model, opt, sched, epoch, dev_loss, best, args)

with open(out/"train_log.json","w") as f: json.dump(history,f,indent=2)
logger.info(f"Training done. Best dev: {best:.4f}")

#
# §13 Inference helpers
#

def load_model(output_dir, logger):
    out = Path(output_dir)
    ckpt = torch.load(out/"best_model.pt", map_location="cpu",
                      weights_only=False)
    cfg = ckpt["model_config"]
    arch = cfg.get("arch","TransformerTTS")
    if arch != "TransformerTTS":
        logger.warning(f"Checkpoint arch={arch}; loading as TransformerTTS_
↳ anyway.")
    model = TransformerTTS(
        vocab_size=cfg["vocab_size"], n_mels=cfg["n_mels"],
        d_model=cfg["d_model"], n_heads=cfg["n_heads"],
        n_enc_layers=cfg["n_layers"], n_dec_layers=cfg["n_layers"],
        d_ff=cfg["ffn_dim"], dropout=cfg.get("dropout",0.1))
    model.load_state_dict(ckpt["model_state"]); model.eval()
    logger.info(f"Loaded TransformerTTS: epoch={ckpt.get('epoch','?')} "
                f"dev={ckpt.get('dev_loss',float('nan')):.4f}")

```



```

return model

def synthesize_text(text, model, vocab, device, speed_factor=1.0,
                    logger=None, mean_fpc=7.3):
    """
    Text → normalized mel via autoregressive inference.
    No oversmoothing: each frame conditioned on previous actual output.

    Length-aware generation (FIX for the "11-frame regardless of text
    length" bug): min_frames/max_frames are computed from the number of
    input tokens × mean_fpc (frames/char from training data), so a long
    sentence cannot stop after 11 frames just because the stop-token
    fired early. This is a safety net around guided_attention_loss(),
    which is the underlying training-time fix for the same problem.
    """
    ids = torch.tensor([vocab.encode(text)], dtype=torch.long, device=device)
    n_tokens = ids.size(1) # incl. BOS/EOS
    expected = max(10, int(round(n_tokens * mean_fpc)))
    min_frames = max(10, int(round(expected * 0.5)))
    max_frames = int(round(expected * 2.5)) + 20

    if logger:
        logger.info(f" length : {n_tokens} tokens × {mean_fpc:.1f} fpc "
                    f" {expected} frames (floor={min_frames}, cap={max_frames})")

    with torch.no_grad():
        mel_norm = model.infer(ids, max_frames=max_frames, min_frames=min_frames)

    if abs(speed_factor-1.0) > 0.01:
        from scipy.ndimage import zoom
        t_new = max(10, round(mel_norm.shape[0]/speed_factor))
        mel_norm = zoom(mel_norm, (t_new/mel_norm.shape[0], 1), order=1)

    mel_db = mel_denormalize(mel_norm)
    if logger:
        logger.info(f" mel_norm : min={mel_norm.min():.3f} "
                    f"max={mel_norm.max():.3f} mean={mel_norm.mean():.3f}")
        logger.info(f" mel_db : min={mel_db.min():.1f} "
                    f"max={mel_db.max():.1f} mean={mel_db.mean():.1f} dB")
        logger.info(f" shape : {mel_norm.shape} "
                    f"({mel_norm.shape[0]*256/22050:.2f}s at default hop/sr)")
    if mel_db.max() < -30.0:
        logger.warning(f" mel_db max={mel_db.max():.1f} dB. "
                       "Model may need more epochs (try 200+).")
    else:

```

```

        logger.info(f"    mel_db max={mel_db.max():.1f} dB - healthy")
    return mel_norm

def _find_gt_id(sid, output_dir, logger):
    for sp in ("train", "dev", "test"):
        for s in _load_split(output_dir, sp):
            if s["id"]==sid:
                logger.info(f"Found {sid} in {sp}")
                return np.load(s["mel"]), s["text"]
    return None, None

def _find_gt_text(text, output_dir, logger):
    for sp in ("train", "dev", "test"):
        for s in _load_split(output_dir, sp):
            if s["text"].strip()==text.strip():
                logger.info(f"Found text match in {sp}")
                return np.load(s["mel"])
    return None

#
# S14 Synthesis stage
#

def run_synth(args, logger):
    out = Path(args.output_dir)
    vocab, mel_cfg = _load_artifacts(args.output_dir)
    mean_fpc = _load_mean_fpc(args.output_dir, logger)
    vocoder = Vocoder(mel_cfg, logger, use_neural=not args.no_neural_vocoder)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    if args.list_samples:
        logger.info("Sample IDs (first 20):")
        for sp in ("train", "dev", "test"):
            items = _load_split(args.output_dir, sp)
            if items:
                for s in items[:20]:
                    logger.info(f"    {s['id']:32s}    {s['text'][:50]}")
                break
        return

    if args.gt_sample_id:
        gm, gt = _find_gt_id(args.gt_sample_id, args.output_dir, logger)
        if gm is None: logger.error(f"Sample '{args.gt_sample_id}' not found.");
    return

```

```

logger.info(f"GT text: {gt}")
db = mel_denormalize(gm)
logger.info(f"GT mel: min={db.min():.1f} max={db.max():.1f} "
            f"mean={db.mean():.1f} dB shape={gm.shape}")
audio = vocoder.synthesize(gm)
wpath = args.synth_out or f"gt_{args.gt_sample_id}.wav"
save_wav(audio, wpath, mel_cfg.sample_rate)
plot_mel(gm, f"GT: {gt[:50]}", str(Path(wpath).with_suffix(".png")),
logger)
logger.info(f"GT audio → {wpath} ({len(audio)/mel_cfg.sample_rate:.
2f}s)")
if not args.gt_compare: return

if args.use_gt_mel and args.synth_text:
    gm = _find_gt_text(args.synth_text, args.output_dir, logger)
    if gm is not None:
        audio = vocoder.synthesize(gm)
        wpath = args.synth_out or "gt_synth.wav"
        save_wav(audio, wpath, mel_cfg.sample_rate)
        plot_mel(gm, f"GT: {args.synth_text[:50]}",
                  str(Path(wpath).with_suffix(".png")), logger)
        logger.info(f"GT-matched audio → {wpath}"); return
    logger.warning("No text match for --use_gt_mel; running model inference.
")

model = load_model(args.output_dir, logger); model.to(device)
text = args.synth_text or "မင်္ဂလာပါ ကျောင်းသားများ"
logger.info(f"Synthesizing: {text!r}")

t0 = time.time()
mel_pred = synthesize_text(text, model, vocab, device,
                           speed_factor=args.speed_factor, logger=logger,
                           mean_fpc=mean_fpc)
logger.info(f"Acoustic model: {(time.time()-t0)*1000:.0f} ms")

wpath = args.synth_out or str(out/"synthesized.wav")
plot_mel(mel_pred, f"TTS v4: {text[:50]}",
          str(Path(wpath).with_suffix(".png")), logger)

t1 = time.time()
audio = vocoder.synthesize(mel_pred)
save_wav(audio, wpath, mel_cfg.sample_rate)
dur = len(audio)/mel_cfg.sample_rate
logger.info(f"Vocoder: {(time.time()-t1)*1000:.0f} ms | "
            f"Audio → {wpath} ({dur:.2f}s)")

if args.gt_compare:

```

```

gm = _find_gt_text(text, args.output_dir, logger)
if gm is not None:
    ga = vocoder.synthesize(gm)
    gw = str(Path(wpath).stem)+"_GT.wav"
    save_wav(ga, gw, mel_cfg.sample_rate)
    logger.info(f"GT comparison → {gw}")
    plot_comparison(mel_pred, gm,
                    str(Path(wpath).stem)+"_compare.png", logger)
else:
    logger.info("No exact GT match found.")

#
# §15 Evaluation
#

def run_eval(args, logger):
    out = Path(args.output_dir)
    vocab, mel_cfg = _load_artifacts(args.output_dir)
    mean_fpc = _load_mean_fpc(args.output_dir, logger)
    model = load_model(args.output_dir, logger)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)

    test_s = _load_split(args.output_dir, "test")
    n_ev = min(args.eval_n, len(test_s))
    if n_ev==0: logger.warning("No test samples."); return

    logger.info(f"Evaluating {n_ev} utterances ...")
    ed = out/"eval_output"; ed.mkdir(exist_ok=True)
    vocoder = Vocoder(mel_cfg, logger, use_neural=not args.no_neural_vocoder)
    results=[]; maes=[]; rtfs=[]; ts=[]

    for i, s in enumerate(test_s[:n_ev]):
        t0=time.time()
        try: mel_pred = synthesize_text(s["text"], model, vocab, device,
                                         mean_fpc=mean_fpc)

        except Exception as e:
            logger.warning(f" [{i+1}/{n_ev}] {s['id']} failed: {e}"); continue
        tt=time.time()-t0; ts.append(tt)

    mae=None
    try:
        gm=np.load(s["mel"]); T=min(mel_pred.shape[0],gm.shape[0])
        mae=float(np.abs(mel_pred[:T]-gm[:T]).mean()); maes.append(mae)
    except: pass

    plot_mel(mel_pred, f"Synth: {s['text'][:40]}",

```

```

        str(ed/f"{s['id']}_mel.png"), logger)

    rtf=None
    try:
        a=vocoder.synthesize(mel_pred); d=len(a)/mel_cfg.sample_rate
        rtf=tt/d if d>0 else None
        save_wav(a, str(ed/f"{s['id']}.wav"), mel_cfg.sample_rate)
        if rtf: rtfs.append(rtf)
    except Exception as e: logger.debug(f"Vocoder: {e}")

    ms=f"{mae:.4f}" if mae else "N/A"; rs=f"{rtf:.3f}" if rtf else "N/A"
    logger.info(f" [{i+1:3d}/{n_ev}] {s['id']} mae={ms} "
                f"synth={tt*1000:.0f}ms RTF={rs}")
    results.append({"id":s["id"],"text":s["text"],"mel_mae":mae,
                   "synth_ms":round(tt*1000,1),"rtf":rtf})

am = float(np.mean(maes)) if maes else float("nan")
ar = float(np.mean(rtfs)) if rtfs else float("nan")
at = float(np.mean(ts))*1000
logger.info(""); logger.info("=*60)
logger.info(" EVALUATION - Myanmar SLR80 TTS Demo v4")
logger.info("=*60)
logger.info(f" Utterances : {n_ev}")
logger.info(f" Avg Mel MAE: {am:.4f}")
logger.info(f" Avg Synth : {at:.1f} ms")
logger.info(f" Avg RTF : {ar:.3f}")
logger.info(" UTMOS: https://github.com/sarulab-speech/UTMOS22")
logger.info("=*60)

with open(ed/"eval_results.json","w",encoding="utf-8") as f:
    json.dump({"summary":{"n_eval":n_ev,"mel_mae":am,"synth_ms":at,"rtf":ar},
              "utterances":results},f,ensure_ascii=False,indent=2)
logger.info(f"Results → {ed/'eval_results.json'}")

#
# §16 CLI
#

def build_parser():
    p = argparse.ArgumentParser(
        description="Myanmar TTS Demo v4 - TransformerTTS (autoregressive)",
        formatter_class=argparse.RawDescriptionHelpFormatter)

    p.add_argument("--stage", choices=["prep","train","synth","eval","all"],
                   default="all")
    p.add_argument("--check_deps", action="store_true")
    p.add_argument("--data_dir", default="./data/slr80")

```

```

p.add_argument("--tsv_file", default="line_index_female.tsv")
p.add_argument("--output_dir", default="./tts_demo")
p.add_argument("--train_ratio", type=float, default=0.80)
p.add_argument("--dev_ratio", type=float, default=0.10)
p.add_argument("--max_samples", type=int, default=0)
p.add_argument("--sample_rate", type=int, default=22050)
p.add_argument("--n_fft", type=int, default=1024)
p.add_argument("--hop_length", type=int, default=256)
p.add_argument("--win_length", type=int, default=1024)
p.add_argument("--n_mels", type=int, default=80)
p.add_argument("--fmin", type=float, default=0.0)
p.add_argument("--fmax", type=float, default=8000.0)
p.add_argument("--d_model", type=int, default=256)
p.add_argument("--n_heads", type=int, default=4)
p.add_argument("--n_layers", type=int, default=4,
                help="Encoder AND decoder layers")
p.add_argument("--ffn_dim", type=int, default=1024)
p.add_argument("--dropout", type=float, default=0.1)
p.add_argument("--epochs", type=int, default=300)
p.add_argument("--warmup_epochs", type=int, default=20)
p.add_argument("--lr", type=float, default=1e-3)
p.add_argument("--batch_size", type=int, default=16)
p.add_argument("--weight_decay", type=float, default=1e-6)
p.add_argument("--grad_clip", type=float, default=1.0)
p.add_argument("--num_workers", type=int, default=2)
p.add_argument("--save_every", type=int, default=10)
p.add_argument("--log_steps", type=int, default=20)
# Backward-compat args (accepted, ignored)
p.add_argument("--dur_loss_weight", type=float, default=0.0,
                help="[DEPRECATED] ignored in v4 (no duration predictor)")
p.add_argument("--synth_text", type=str,
                default="မင်္ဂလာပါ ကျောင်းသားများ")
p.add_argument("--synth_out", type=str, default=None)
p.add_argument("--speed_factor", type=float, default=1.0,
                help="0.8=slower 1.0=normal 1.2=faster")
p.add_argument("--eval_n", type=int, default=50)
p.add_argument("--log_file", default=None)
p.add_argument("--verbose", action="store_true")
p.add_argument("--seed", type=int, default=42)
p.add_argument("--resume", type=str, default=None)
p.add_argument("--list_samples", action="store_true")
p.add_argument("--gt_sample_id", type=str, default=None)
p.add_argument("--use_gt_mel", action="store_true")
p.add_argument("--gt_compare", action="store_true")
p.add_argument("--no_neural_vocoder", action="store_true")
return p

```

```

def main():
    parser = build_parser(); args = parser.parse_args()
    logger = setup_logging(args.log_file, args.verbose)

    if args.check_deps:
        sys.exit(0 if check_dependencies(logger) else 1)

    random.seed(args.seed); np.random.seed(args.seed)
    torch.manual_seed(args.seed)
    if torch.cuda.is_available(): torch.cuda.manual_seed_all(args.seed)

    mel_cfg = MelConfig(sample_rate=args.sample_rate, n_fft=args.n_fft,
                        hop_length=args.hop_length, win_length=args.win_length,
                        n_mels=args.n_mels, fmin=args.fmin, fmax=args.fmax)

    if args.stage in ("prep", "all"):
        prepare_data(args.data_dir, args.tsv_file, args.output_dir,
                    args.train_ratio, args.dev_ratio, args.seed,
                    mel_cfg, args.max_samples, logger)
    if args.stage in ("train", "all"): run_train(args, logger)
    if args.stage in ("synth", "all"): run_synth(args, logger)
    if args.stage in ("eval", "all"): run_eval(args, logger)

if __name__ == "__main__":
    main()

```

0.4 Running Steps

1 Data Preparation

```
python myanmar_tts_transformer.py -stage prep -data_dir ./data/slr80 -output_dir
./tts_output
```

2 Train from scratch — delete old best_model.pt first

```
rm -f ./tts_output/best_model.pt ./tts_output/checkpoint_*.pt
```

```
python myanmar_tts_transformer.py -stage train
-output_dir ./tts_output -epochs 300
```

3 Vocoder sanity check (should still sound perfect)

```
python myanmar_tts_transformer.py -stage synth
-output_dir ./tts_output -gt_sample_id bur_5362_4875790905
```

4 Full TTS — should now produce real Burmese speech, not wind

```
python myanmar_tts_transformer.py --stage synth
--output_dir ./tts_output
--synth_text “ကျွန်တော် ကလည်း ဒိတ်ဒိတ်ကျ အောင်မြင်နေတဲ့ မင်းသား မှ မဟုတ်တာလေ”
--gt_compare
```

[24]: %pwd

[24]: '/mnt/disk1/ye/exp/asr_tts_SLR80'

4.1 Step1: Data Preparation

[25]: !time python myanmar_tts_transformer.py --stage prep \N
--data_dir ./data/slr80 --output_dir ./tts_output

```
2026-06-19 14:07:38 [INFO    ] Found 2530 audio files.
2026-06-19 14:07:38 [INFO    ] Vocabulary: 66 characters
2026-06-19 14:07:38 [INFO    ] Computing mel spectrograms ...
2026-06-19 14:08:00 [INFO    ] 500/2530 ...
2026-06-19 14:08:23 [INFO    ] 1000/2530 ...
2026-06-19 14:08:45 [INFO    ] 1500/2530 ...
2026-06-19 14:08:57 [INFO    ] 2000/2530 ...
2026-06-19 14:09:01 [INFO    ] 2500/2530 ...
2026-06-19 14:09:01 [INFO    ] Valid: 2530 | Skipped: 0
2026-06-19 14:09:01 [INFO    ] Mel norm check: min=-1.000 max=1.000
mean=-0.481
2026-06-19 14:09:01 [INFO    ] Mel frames : mean=504 ± 150
2026-06-19 14:09:01 [INFO    ] Frames/char: mean=7.3 ± 1.6
2026-06-19 14:09:02 [INFO    ] train: 2024
2026-06-19 14:09:02 [INFO    ] dev: 253
2026-06-19 14:09:02 [INFO    ] test: 253
2026-06-19 14:09:02 [INFO    ] Data preparation complete

real    1m24.787s
user    30m44.848s
sys      0m3.210s
```

4.2 Step2: Training

[26]: !time python myanmar_tts_transformer.py --stage train \N
--output_dir ./tts_output --epochs 300

```
2026-06-19 14:14:06 [INFO    ] Vocab:66 Train:2024 Dev:253
2026-06-19 14:14:06 [INFO    ] Device: cuda
2026-06-19 14:14:06 [INFO    ] TransformerTTS parameters: 45,399,617
2026-06-19 14:14:07 [INFO    ] Training 300 epochs (warmup=20, lr=1.0e-03)
2026-06-19 14:14:07 [INFO    ] Architecture: TransformerTTS + Guided Attention
Loss (v5)
```



```

2026-06-19 14:14:07 [INFO ] Expected: ep1 0.8-1.1 ep50 0.35-0.55
ep200 0.10-0.25
2026-06-19 14:14:07 [INFO ] v4 bug: stop fired ~11 frames regardless of
text length (exposure
2026-06-19 14:14:07 [INFO ] bias + unconstrained attention). Guided
attention loss forces
2026-06-19 14:14:07 [INFO ] monotonic left-to-right alignment, fixing the
early-stop collapse.
2026-06-19 14:14:26 [INFO ] Epoch 1/300 | train=1.7123 dev=1.0000 |
lr=1.00e-04
2026-06-19 14:14:26 [INFO ] Best model saved (dev=1.0000)
2026-06-19 14:14:44 [INFO ] Epoch 2/300 | train=1.0673 dev=0.8140 |
lr=1.50e-04
2026-06-19 14:14:47 [INFO ] Best model saved (dev=0.8140)
2026-06-19 14:15:05 [INFO ] Epoch 3/300 | train=0.9539 dev=0.7807 |
lr=2.00e-04
2026-06-19 14:15:08 [INFO ] Best model saved (dev=0.7807)
2026-06-19 14:15:27 [INFO ] Epoch 4/300 | train=0.8213 dev=0.6477 |
lr=2.50e-04
2026-06-19 14:15:29 [INFO ] Best model saved (dev=0.6477)
2026-06-19 14:15:48 [INFO ] Epoch 5/300 | train=0.7255 dev=0.6196 |
lr=3.00e-04
2026-06-19 14:15:51 [INFO ] Best model saved (dev=0.6196)
2026-06-19 14:16:09 [INFO ] Epoch 6/300 | train=0.9008 dev=0.8662 |
lr=3.50e-04
2026-06-19 14:16:27 [INFO ] Epoch 7/300 | train=0.9117 dev=0.7261 |
lr=4.00e-04
2026-06-19 14:16:46 [INFO ] Epoch 8/300 | train=0.7125 dev=0.5920 |
lr=4.50e-04
2026-06-19 14:16:49 [INFO ] Best model saved (dev=0.5920)
2026-06-19 14:17:07 [INFO ] Epoch 9/300 | train=0.7135 dev=0.6522 |
lr=5.00e-04
2026-06-19 14:17:26 [INFO ] Epoch 10/300 | train=0.6325 dev=0.5550 |
lr=5.50e-04
2026-06-19 14:17:28 [INFO ] Best model saved (dev=0.5550)
2026-06-19 14:17:47 [INFO ] Epoch 11/300 | train=0.5830 dev=0.5309 |
lr=6.00e-04
2026-06-19 14:17:50 [INFO ] Best model saved (dev=0.5309)
2026-06-19 14:18:08 [INFO ] Epoch 12/300 | train=0.5561 dev=0.5048 |
lr=6.50e-04
2026-06-19 14:18:11 [INFO ] Best model saved (dev=0.5048)
2026-06-19 14:18:30 [INFO ] Epoch 13/300 | train=0.5287 dev=0.4841 |
lr=7.00e-04
2026-06-19 14:18:32 [INFO ] Best model saved (dev=0.4841)
2026-06-19 14:18:51 [INFO ] Epoch 14/300 | train=0.4971 dev=0.4894 |
lr=7.50e-04
2026-06-19 14:19:10 [INFO ] Epoch 15/300 | train=0.4620 dev=0.4364 |
lr=8.00e-04

```

```

2026-06-19 14:19:12 [INFO ] Best model saved (dev=0.4364)
2026-06-19 14:19:31 [INFO ] Epoch 16/300 | train=0.4506 dev=0.4218 |
lr=8.50e-04
2026-06-19 14:19:34 [INFO ] Best model saved (dev=0.4218)
2026-06-19 14:19:52 [INFO ] Epoch 17/300 | train=0.4516 dev=0.4185 |
lr=9.00e-04
2026-06-19 14:19:55 [INFO ] Best model saved (dev=0.4185)
2026-06-19 14:20:14 [INFO ] Epoch 18/300 | train=0.4188 dev=0.4038 |
lr=9.50e-04
2026-06-19 14:20:16 [INFO ] Best model saved (dev=0.4038)
2026-06-19 14:20:35 [INFO ] Epoch 19/300 | train=0.4102 dev=0.3912 |
lr=1.00e-03
2026-06-19 14:20:38 [INFO ] Best model saved (dev=0.3912)
2026-06-19 14:20:56 [INFO ] Epoch 20/300 | train=0.3986 dev=0.3885 |
lr=1.00e-03
2026-06-19 14:20:59 [INFO ] Best model saved (dev=0.3885)
2026-06-19 14:21:18 [INFO ] Epoch 21/300 | train=0.3956 dev=0.3731 |
lr=1.00e-03
2026-06-19 14:21:20 [INFO ] Best model saved (dev=0.3731)
2026-06-19 14:21:39 [INFO ] Epoch 22/300 | train=0.3886 dev=0.3669 |
lr=1.00e-03
2026-06-19 14:21:42 [INFO ] Best model saved (dev=0.3669)
2026-06-19 14:22:00 [INFO ] Epoch 23/300 | train=0.3771 dev=0.3643 |
lr=1.00e-03
2026-06-19 14:22:03 [INFO ] Best model saved (dev=0.3643)
2026-06-19 14:22:21 [INFO ] Epoch 24/300 | train=0.3710 dev=0.3574 |
lr=1.00e-03
2026-06-19 14:22:24 [INFO ] Best model saved (dev=0.3574)
2026-06-19 14:22:43 [INFO ] Epoch 25/300 | train=0.3658 dev=0.3603 |
lr=9.99e-04
2026-06-19 14:23:01 [INFO ] Epoch 26/300 | train=0.3620 dev=0.3537 |
lr=9.99e-04
2026-06-19 14:23:04 [INFO ] Best model saved (dev=0.3537)
2026-06-19 14:23:23 [INFO ] Epoch 27/300 | train=0.3587 dev=0.3429 |
lr=9.99e-04
2026-06-19 14:23:25 [INFO ] Best model saved (dev=0.3429)
2026-06-19 14:23:44 [INFO ] Epoch 28/300 | train=0.3529 dev=0.3385 |
lr=9.98e-04
2026-06-19 14:23:47 [INFO ] Best model saved (dev=0.3385)
2026-06-19 14:24:05 [INFO ] Epoch 29/300 | train=0.3477 dev=0.3350 |
lr=9.98e-04
2026-06-19 14:24:08 [INFO ] Best model saved (dev=0.3350)
2026-06-19 14:24:27 [INFO ] Epoch 30/300 | train=0.3491 dev=0.3369 |
lr=9.97e-04
2026-06-19 14:24:45 [INFO ] Epoch 31/300 | train=0.3395 dev=0.3276 |
lr=9.96e-04
2026-06-19 14:24:48 [INFO ] Best model saved (dev=0.3276)
2026-06-19 14:25:07 [INFO ] Epoch 32/300 | train=0.3382 dev=0.3228 |

```

```

lr=9.96e-04
2026-06-19 14:25:10 [INFO ] Best model saved (dev=0.3228)
2026-06-19 14:25:28 [INFO ] Epoch 33/300 | train=0.3379 dev=0.3393 |
lr=9.95e-04
2026-06-19 14:25:47 [INFO ] Epoch 34/300 | train=0.3375 dev=0.3241 |
lr=9.94e-04
2026-06-19 14:26:05 [INFO ] Epoch 35/300 | train=0.3301 dev=0.3182 |
lr=9.93e-04
2026-06-19 14:26:08 [INFO ] Best model saved (dev=0.3182)
2026-06-19 14:26:27 [INFO ] Epoch 36/300 | train=0.3339 dev=0.3167 |
lr=9.92e-04
2026-06-19 14:26:30 [INFO ] Best model saved (dev=0.3167)
2026-06-19 14:26:48 [INFO ] Epoch 37/300 | train=0.3299 dev=0.3232 |
lr=9.91e-04
2026-06-19 14:27:06 [INFO ] Epoch 38/300 | train=0.3270 dev=0.3149 |
lr=9.90e-04
2026-06-19 14:27:09 [INFO ] Best model saved (dev=0.3149)
2026-06-19 14:27:28 [INFO ] Epoch 39/300 | train=0.3259 dev=0.3138 |
lr=9.89e-04
2026-06-19 14:27:31 [INFO ] Best model saved (dev=0.3138)
2026-06-19 14:27:49 [INFO ] Epoch 40/300 | train=0.3265 dev=0.3132 |
lr=9.88e-04
2026-06-19 14:27:52 [INFO ] Best model saved (dev=0.3132)
2026-06-19 14:28:11 [INFO ] Epoch 41/300 | train=0.3230 dev=0.3117 |
lr=9.87e-04
2026-06-19 14:28:14 [INFO ] Best model saved (dev=0.3117)
2026-06-19 14:28:32 [INFO ] Epoch 42/300 | train=0.3221 dev=0.3107 |
lr=9.86e-04
2026-06-19 14:28:35 [INFO ] Best model saved (dev=0.3107)
2026-06-19 14:28:54 [INFO ] Epoch 43/300 | train=0.3214 dev=0.3102 |
lr=9.84e-04
2026-06-19 14:28:57 [INFO ] Best model saved (dev=0.3102)
2026-06-19 14:29:15 [INFO ] Epoch 44/300 | train=0.3207 dev=0.3134 |
lr=9.83e-04
2026-06-19 14:29:34 [INFO ] Epoch 45/300 | train=0.3155 dev=0.3056 |
lr=9.81e-04
2026-06-19 14:29:37 [INFO ] Best model saved (dev=0.3056)
2026-06-19 14:29:55 [INFO ] Epoch 46/300 | train=0.3181 dev=0.3050 |
lr=9.80e-04
2026-06-19 14:29:58 [INFO ] Best model saved (dev=0.3050)
2026-06-19 14:30:17 [INFO ] Epoch 47/300 | train=0.3163 dev=0.3033 |
lr=9.78e-04
2026-06-19 14:30:19 [INFO ] Best model saved (dev=0.3033)
2026-06-19 14:30:38 [INFO ] Epoch 48/300 | train=0.3146 dev=0.3055 |
lr=9.77e-04
2026-06-19 14:30:57 [INFO ] Epoch 49/300 | train=0.3122 dev=0.3014 |
lr=9.75e-04
2026-06-19 14:30:59 [INFO ] Best model saved (dev=0.3014)

```

```

2026-06-19 14:31:18 [INFO ] Epoch 50/300 | train=0.3126 dev=0.3037 |
lr=9.73e-04
2026-06-19 14:31:37 [INFO ] Epoch 51/300 | train=0.3089 dev=0.3000 |
lr=9.72e-04
2026-06-19 14:31:42 [INFO ] Best model saved (dev=0.3000)
2026-06-19 14:32:00 [INFO ] Epoch 52/300 | train=0.3090 dev=0.3037 |
lr=9.70e-04
2026-06-19 14:32:19 [INFO ] Epoch 53/300 | train=0.3070 dev=0.3030 |
lr=9.68e-04
2026-06-19 14:32:37 [INFO ] Epoch 54/300 | train=0.3052 dev=0.2968 |
lr=9.66e-04
2026-06-19 14:32:40 [INFO ] Best model saved (dev=0.2968)
2026-06-19 14:32:59 [INFO ] Epoch 55/300 | train=0.3064 dev=0.3001 |
lr=9.64e-04
2026-06-19 14:33:17 [INFO ] Epoch 56/300 | train=0.3053 dev=0.2941 |
lr=9.62e-04
2026-06-19 14:33:20 [INFO ] Best model saved (dev=0.2941)
2026-06-19 14:33:38 [INFO ] Epoch 57/300 | train=0.3025 dev=0.2918 |
lr=9.60e-04
2026-06-19 14:33:41 [INFO ] Best model saved (dev=0.2918)
2026-06-19 14:34:00 [INFO ] Epoch 58/300 | train=0.3034 dev=0.2951 |
lr=9.57e-04
2026-06-19 14:34:18 [INFO ] Epoch 59/300 | train=0.3004 dev=0.2904 |
lr=9.55e-04
2026-06-19 14:34:21 [INFO ] Best model saved (dev=0.2904)
2026-06-19 14:34:39 [INFO ] Epoch 60/300 | train=0.3013 dev=0.2910 |
lr=9.53e-04
2026-06-19 14:34:58 [INFO ] Epoch 61/300 | train=0.3031 dev=0.2920 |
lr=9.51e-04
2026-06-19 14:35:17 [INFO ] Epoch 62/300 | train=0.2988 dev=0.2869 |
lr=9.48e-04
2026-06-19 14:35:19 [INFO ] Best model saved (dev=0.2869)
2026-06-19 14:35:38 [INFO ] Epoch 63/300 | train=0.2966 dev=0.3024 |
lr=9.46e-04
2026-06-19 14:35:56 [INFO ] Epoch 64/300 | train=0.2996 dev=0.2915 |
lr=9.43e-04
2026-06-19 14:36:15 [INFO ] Epoch 65/300 | train=0.2966 dev=0.2952 |
lr=9.41e-04
2026-06-19 14:36:33 [INFO ] Epoch 66/300 | train=0.2949 dev=0.2839 |
lr=9.38e-04
2026-06-19 14:36:36 [INFO ] Best model saved (dev=0.2839)
2026-06-19 14:36:54 [INFO ] Epoch 67/300 | train=0.2941 dev=0.2878 |
lr=9.35e-04
2026-06-19 14:37:13 [INFO ] Epoch 68/300 | train=0.2922 dev=0.2833 |
lr=9.33e-04
2026-06-19 14:37:16 [INFO ] Best model saved (dev=0.2833)
2026-06-19 14:37:34 [INFO ] Epoch 69/300 | train=0.2959 dev=0.2852 |
lr=9.30e-04

```

```

2026-06-19 14:37:53 [INFO ] Epoch 70/300 | train=0.2944 dev=0.2813 |
lr=9.27e-04
2026-06-19 14:37:56 [INFO ] Best model saved (dev=0.2813)
2026-06-19 14:38:14 [INFO ] Epoch 71/300 | train=0.2928 dev=0.2870 |
lr=9.24e-04
2026-06-19 14:38:33 [INFO ] Epoch 72/300 | train=0.2909 dev=0.2796 |
lr=9.21e-04
2026-06-19 14:38:36 [INFO ] Best model saved (dev=0.2796)
2026-06-19 14:38:54 [INFO ] Epoch 73/300 | train=0.2915 dev=0.2856 |
lr=9.18e-04
2026-06-19 14:39:13 [INFO ] Epoch 74/300 | train=0.2894 dev=0.2825 |
lr=9.15e-04
2026-06-19 14:39:31 [INFO ] Epoch 75/300 | train=0.2935 dev=0.2824 |
lr=9.12e-04
2026-06-19 14:39:50 [INFO ] Epoch 76/300 | train=0.2885 dev=0.2798 |
lr=9.09e-04
2026-06-19 14:40:08 [INFO ] Epoch 77/300 | train=0.2871 dev=0.2786 |
lr=9.06e-04
2026-06-19 14:40:11 [INFO ] Best model saved (dev=0.2786)
2026-06-19 14:40:30 [INFO ] Epoch 78/300 | train=0.2903 dev=0.2809 |
lr=9.03e-04
2026-06-19 14:40:48 [INFO ] Epoch 79/300 | train=0.2883 dev=0.2780 |
lr=9.00e-04
2026-06-19 14:40:51 [INFO ] Best model saved (dev=0.2780)
2026-06-19 14:41:10 [INFO ] Epoch 80/300 | train=0.2863 dev=0.2870 |
lr=8.96e-04
2026-06-19 14:41:29 [INFO ] Epoch 81/300 | train=0.2853 dev=0.2740 |
lr=8.93e-04
2026-06-19 14:41:32 [INFO ] Best model saved (dev=0.2740)
2026-06-19 14:41:50 [INFO ] Epoch 82/300 | train=0.2877 dev=0.2764 |
lr=8.90e-04
2026-06-19 14:42:09 [INFO ] Epoch 83/300 | train=0.2824 dev=0.2746 |
lr=8.86e-04
2026-06-19 14:42:28 [INFO ] Epoch 84/300 | train=0.2846 dev=0.2778 |
lr=8.83e-04
2026-06-19 14:42:46 [INFO ] Epoch 85/300 | train=0.2827 dev=0.2794 |
lr=8.79e-04
2026-06-19 14:43:05 [INFO ] Epoch 86/300 | train=0.2883 dev=0.2782 |
lr=8.76e-04
2026-06-19 14:43:23 [INFO ] Epoch 87/300 | train=0.2824 dev=0.2779 |
lr=8.72e-04
2026-06-19 14:43:42 [INFO ] Epoch 88/300 | train=0.2814 dev=0.2733 |
lr=8.68e-04
2026-06-19 14:43:45 [INFO ] Best model saved (dev=0.2733)
2026-06-19 14:44:03 [INFO ] Epoch 89/300 | train=0.2812 dev=0.2733 |
lr=8.65e-04
2026-06-19 14:44:22 [INFO ] Epoch 90/300 | train=0.2819 dev=0.2793 |
lr=8.61e-04

```

```

2026-06-19 14:44:41 [INFO ] Epoch 91/300 | train=0.2810 dev=0.2755 |
lr=8.57e-04
2026-06-19 14:44:59 [INFO ] Epoch 92/300 | train=0.2816 dev=0.2761 |
lr=8.53e-04
2026-06-19 14:45:17 [INFO ] Epoch 93/300 | train=0.2800 dev=0.2695 |
lr=8.49e-04
2026-06-19 14:45:20 [INFO ] Best model saved (dev=0.2695)
2026-06-19 14:45:39 [INFO ] Epoch 94/300 | train=0.2781 dev=0.2699 |
lr=8.45e-04
2026-06-19 14:45:57 [INFO ] Epoch 95/300 | train=0.2813 dev=0.2707 |
lr=8.42e-04
2026-06-19 14:46:16 [INFO ] Epoch 96/300 | train=0.2808 dev=0.2721 |
lr=8.38e-04
2026-06-19 14:46:34 [INFO ] Epoch 97/300 | train=0.2764 dev=0.2682 |
lr=8.33e-04
2026-06-19 14:46:37 [INFO ] Best model saved (dev=0.2682)
2026-06-19 14:46:56 [INFO ] Epoch 98/300 | train=0.2821 dev=0.2727 |
lr=8.29e-04
2026-06-19 14:47:14 [INFO ] Epoch 99/300 | train=0.2762 dev=0.2683 |
lr=8.25e-04
2026-06-19 14:47:33 [INFO ] Epoch 100/300 | train=0.2756 dev=0.2771 |
lr=8.21e-04
2026-06-19 14:47:51 [INFO ] Epoch 101/300 | train=0.2768 dev=0.2719 |
lr=8.17e-04
2026-06-19 14:48:10 [INFO ] Epoch 102/300 | train=0.2760 dev=0.2666 |
lr=8.13e-04
2026-06-19 14:48:13 [INFO ] Best model saved (dev=0.2666)
2026-06-19 14:48:31 [INFO ] Epoch 103/300 | train=0.2766 dev=0.2681 |
lr=8.08e-04
2026-06-19 14:48:50 [INFO ] Epoch 104/300 | train=0.2733 dev=0.2656 |
lr=8.04e-04
2026-06-19 14:48:53 [INFO ] Best model saved (dev=0.2656)
2026-06-19 14:49:11 [INFO ] Epoch 105/300 | train=0.2766 dev=0.2702 |
lr=8.00e-04
2026-06-19 14:49:30 [INFO ] Epoch 106/300 | train=0.2738 dev=0.2683 |
lr=7.96e-04
2026-06-19 14:49:48 [INFO ] Epoch 107/300 | train=0.2745 dev=0.2656 |
lr=7.91e-04
2026-06-19 14:50:07 [INFO ] Epoch 108/300 | train=0.2763 dev=0.2654 |
lr=7.87e-04
2026-06-19 14:50:10 [INFO ] Best model saved (dev=0.2654)
2026-06-19 14:50:28 [INFO ] Epoch 109/300 | train=0.2760 dev=0.2650 |
lr=7.82e-04
2026-06-19 14:50:31 [INFO ] Best model saved (dev=0.2650)
2026-06-19 14:50:50 [INFO ] Epoch 110/300 | train=0.2733 dev=0.2635 |
lr=7.78e-04
2026-06-19 14:50:53 [INFO ] Best model saved (dev=0.2635)
2026-06-19 14:51:11 [INFO ] Epoch 111/300 | train=0.2715 dev=0.2652 |

```

```

lr=7.73e-04
2026-06-19 14:51:30 [INFO ] Epoch 112/300 | train=0.2724 dev=0.2641 |
lr=7.69e-04
2026-06-19 14:51:48 [INFO ] Epoch 113/300 | train=0.2737 dev=0.2671 |
lr=7.64e-04
2026-06-19 14:52:07 [INFO ] Epoch 114/300 | train=0.2707 dev=0.2692 |
lr=7.59e-04
2026-06-19 14:52:25 [INFO ] Epoch 115/300 | train=0.2738 dev=0.2644 |
lr=7.55e-04
2026-06-19 14:52:44 [INFO ] Epoch 116/300 | train=0.2692 dev=0.2618 |
lr=7.50e-04
2026-06-19 14:52:47 [INFO ] Best model saved (dev=0.2618)
2026-06-19 14:53:06 [INFO ] Epoch 117/300 | train=0.2716 dev=0.2661 |
lr=7.45e-04
2026-06-19 14:53:24 [INFO ] Epoch 118/300 | train=0.2693 dev=0.2624 |
lr=7.41e-04
2026-06-19 14:53:42 [INFO ] Epoch 119/300 | train=0.2719 dev=0.2641 |
lr=7.36e-04
2026-06-19 14:54:01 [INFO ] Epoch 120/300 | train=0.2734 dev=0.2615 |
lr=7.31e-04
2026-06-19 14:54:04 [INFO ] Best model saved (dev=0.2615)
2026-06-19 14:54:23 [INFO ] Epoch 121/300 | train=0.2695 dev=0.2620 |
lr=7.26e-04
2026-06-19 14:54:41 [INFO ] Epoch 122/300 | train=0.2682 dev=0.2636 |
lr=7.21e-04
2026-06-19 14:54:59 [INFO ] Epoch 123/300 | train=0.2699 dev=0.2622 |
lr=7.17e-04
2026-06-19 14:55:18 [INFO ] Epoch 124/300 | train=0.2687 dev=0.2648 |
lr=7.12e-04
2026-06-19 14:55:36 [INFO ] Epoch 125/300 | train=0.2698 dev=0.2596 |
lr=7.07e-04
2026-06-19 14:55:39 [INFO ] Best model saved (dev=0.2596)
2026-06-19 14:55:58 [INFO ] Epoch 126/300 | train=0.2695 dev=0.2667 |
lr=7.02e-04
2026-06-19 14:56:16 [INFO ] Epoch 127/300 | train=0.2685 dev=0.2603 |
lr=6.97e-04
2026-06-19 14:56:35 [INFO ] Epoch 128/300 | train=0.2669 dev=0.2623 |
lr=6.92e-04
2026-06-19 14:56:53 [INFO ] Epoch 129/300 | train=0.2668 dev=0.2585 |
lr=6.87e-04
2026-06-19 14:56:56 [INFO ] Best model saved (dev=0.2585)
2026-06-19 14:57:15 [INFO ] Epoch 130/300 | train=0.2687 dev=0.2590 |
lr=6.82e-04
2026-06-19 14:57:34 [INFO ] Epoch 131/300 | train=0.2669 dev=0.2623 |
lr=6.77e-04
2026-06-19 14:57:52 [INFO ] Epoch 132/300 | train=0.2670 dev=0.2584 |
lr=6.72e-04
2026-06-19 14:57:55 [INFO ] Best model saved (dev=0.2584)

```

```

2026-06-19 14:58:14 [INFO ] Epoch 133/300 | train=0.2652 dev=0.2624 |
lr=6.67e-04
2026-06-19 14:58:32 [INFO ] Epoch 134/300 | train=0.2677 dev=0.2613 |
lr=6.62e-04
2026-06-19 14:58:51 [INFO ] Epoch 135/300 | train=0.2649 dev=0.2584 |
lr=6.56e-04
2026-06-19 14:58:54 [INFO ] Best model saved (dev=0.2584)
2026-06-19 14:59:13 [INFO ] Epoch 136/300 | train=0.2688 dev=0.2592 |
lr=6.51e-04
2026-06-19 14:59:31 [INFO ] Epoch 137/300 | train=0.2642 dev=0.2577 |
lr=6.46e-04
2026-06-19 14:59:34 [INFO ] Best model saved (dev=0.2577)
2026-06-19 14:59:53 [INFO ] Epoch 138/300 | train=0.2667 dev=0.2621 |
lr=6.41e-04
2026-06-19 15:00:11 [INFO ] Epoch 139/300 | train=0.2650 dev=0.2587 |
lr=6.36e-04
2026-06-19 15:00:30 [INFO ] Epoch 140/300 | train=0.2643 dev=0.2578 |
lr=6.31e-04
2026-06-19 15:00:49 [INFO ] Epoch 141/300 | train=0.2648 dev=0.2563 |
lr=6.25e-04
2026-06-19 15:00:52 [INFO ] Best model saved (dev=0.2563)
2026-06-19 15:01:10 [INFO ] Epoch 142/300 | train=0.2627 dev=0.2564 |
lr=6.20e-04
2026-06-19 15:01:29 [INFO ] Epoch 143/300 | train=0.2665 dev=0.2563 |
lr=6.15e-04
2026-06-19 15:01:31 [INFO ] Best model saved (dev=0.2563)
2026-06-19 15:01:50 [INFO ] Epoch 144/300 | train=0.2622 dev=0.2561 |
lr=6.10e-04
2026-06-19 15:01:53 [INFO ] Best model saved (dev=0.2561)
2026-06-19 15:02:11 [INFO ] Epoch 145/300 | train=0.2633 dev=0.2573 |
lr=6.05e-04
2026-06-19 15:02:30 [INFO ] Epoch 146/300 | train=0.2622 dev=0.2550 |
lr=5.99e-04
2026-06-19 15:02:33 [INFO ] Best model saved (dev=0.2550)
2026-06-19 15:02:51 [INFO ] Epoch 147/300 | train=0.2632 dev=0.2559 |
lr=5.94e-04
2026-06-19 15:03:10 [INFO ] Epoch 148/300 | train=0.2621 dev=0.2607 |
lr=5.89e-04
2026-06-19 15:03:28 [INFO ] Epoch 149/300 | train=0.2658 dev=0.2569 |
lr=5.83e-04
2026-06-19 15:03:47 [INFO ] Epoch 150/300 | train=0.2613 dev=0.2544 |
lr=5.78e-04
2026-06-19 15:03:50 [INFO ] Best model saved (dev=0.2544)
2026-06-19 15:04:08 [INFO ] Epoch 151/300 | train=0.2608 dev=0.2544 |
lr=5.73e-04
2026-06-19 15:04:27 [INFO ] Epoch 152/300 | train=0.2616 dev=0.2552 |
lr=5.68e-04
2026-06-19 15:04:46 [INFO ] Epoch 153/300 | train=0.2608 dev=0.2552 |

```



```

lr=5.62e-04
2026-06-19 15:05:04 [INFO ] Epoch 154/300 | train=0.2603 dev=0.2586 |
lr=5.57e-04
2026-06-19 15:05:22 [INFO ] Epoch 155/300 | train=0.2607 dev=0.2549 |
lr=5.52e-04
2026-06-19 15:05:41 [INFO ] Epoch 156/300 | train=0.2609 dev=0.2558 |
lr=5.46e-04
2026-06-19 15:06:00 [INFO ] Epoch 157/300 | train=0.2631 dev=0.2533 |
lr=5.41e-04
2026-06-19 15:06:02 [INFO ] Best model saved (dev=0.2533)
2026-06-19 15:06:21 [INFO ] Epoch 158/300 | train=0.2597 dev=0.2550 |
lr=5.36e-04
2026-06-19 15:06:40 [INFO ] Epoch 159/300 | train=0.2598 dev=0.2602 |
lr=5.30e-04
2026-06-19 15:06:58 [INFO ] Epoch 160/300 | train=0.2604 dev=0.2529 |
lr=5.25e-04
2026-06-19 15:07:01 [INFO ] Best model saved (dev=0.2529)
2026-06-19 15:07:20 [INFO ] Epoch 161/300 | train=0.2592 dev=0.2553 |
lr=5.20e-04
2026-06-19 15:07:38 [INFO ] Epoch 162/300 | train=0.2601 dev=0.2525 |
lr=5.14e-04
2026-06-19 15:07:41 [INFO ] Best model saved (dev=0.2525)
2026-06-19 15:08:00 [INFO ] Epoch 163/300 | train=0.2585 dev=0.2535 |
lr=5.09e-04
2026-06-19 15:08:18 [INFO ] Epoch 164/300 | train=0.2611 dev=0.2686 |
lr=5.04e-04
2026-06-19 15:08:37 [INFO ] Epoch 165/300 | train=0.2595 dev=0.2520 |
lr=4.98e-04
2026-06-19 15:08:40 [INFO ] Best model saved (dev=0.2520)
2026-06-19 15:08:58 [INFO ] Epoch 166/300 | train=0.2585 dev=0.2531 |
lr=4.93e-04
2026-06-19 15:09:17 [INFO ] Epoch 167/300 | train=0.2586 dev=0.2514 |
lr=4.88e-04
2026-06-19 15:09:20 [INFO ] Best model saved (dev=0.2514)
2026-06-19 15:09:38 [INFO ] Epoch 168/300 | train=0.2594 dev=0.2543 |
lr=4.82e-04
2026-06-19 15:09:57 [INFO ] Epoch 169/300 | train=0.2592 dev=0.2520 |
lr=4.77e-04
2026-06-19 15:10:15 [INFO ] Epoch 170/300 | train=0.2580 dev=0.2547 |
lr=4.72e-04
2026-06-19 15:10:34 [INFO ] Epoch 171/300 | train=0.2570 dev=0.2515 |
lr=4.67e-04
2026-06-19 15:10:53 [INFO ] Epoch 172/300 | train=0.2577 dev=0.2511 |
lr=4.61e-04
2026-06-19 15:10:56 [INFO ] Best model saved (dev=0.2511)
2026-06-19 15:11:14 [INFO ] Epoch 173/300 | train=0.2571 dev=0.2511 |
lr=4.56e-04
2026-06-19 15:11:33 [INFO ] Epoch 174/300 | train=0.2562 dev=0.2499 |

```

```

lr=4.51e-04
2026-06-19 15:11:35 [INFO ] Best model saved (dev=0.2499)
2026-06-19 15:11:54 [INFO ] Epoch 175/300 | train=0.2566 dev=0.2526 |
lr=4.45e-04
2026-06-19 15:12:13 [INFO ] Epoch 176/300 | train=0.2582 dev=0.2502 |
lr=4.40e-04
2026-06-19 15:12:31 [INFO ] Epoch 177/300 | train=0.2559 dev=0.2503 |
lr=4.35e-04
2026-06-19 15:12:50 [INFO ] Epoch 178/300 | train=0.2556 dev=0.2507 |
lr=4.30e-04
2026-06-19 15:13:09 [INFO ] Epoch 179/300 | train=0.2559 dev=0.2497 |
lr=4.25e-04
2026-06-19 15:13:11 [INFO ] Best model saved (dev=0.2497)
2026-06-19 15:13:30 [INFO ] Epoch 180/300 | train=0.2562 dev=0.2498 |
lr=4.19e-04
2026-06-19 15:13:49 [INFO ] Epoch 181/300 | train=0.2548 dev=0.2486 |
lr=4.14e-04
2026-06-19 15:13:52 [INFO ] Best model saved (dev=0.2486)
2026-06-19 15:14:10 [INFO ] Epoch 182/300 | train=0.2572 dev=0.2494 |
lr=4.09e-04
2026-06-19 15:14:28 [INFO ] Epoch 183/300 | train=0.2546 dev=0.2501 |
lr=4.04e-04
2026-06-19 15:14:47 [INFO ] Epoch 184/300 | train=0.2551 dev=0.2493 |
lr=3.99e-04
2026-06-19 15:15:06 [INFO ] Epoch 185/300 | train=0.2543 dev=0.2496 |
lr=3.94e-04
2026-06-19 15:15:24 [INFO ] Epoch 186/300 | train=0.2550 dev=0.2504 |
lr=3.88e-04
2026-06-19 15:15:43 [INFO ] Epoch 187/300 | train=0.2552 dev=0.2494 |
lr=3.83e-04
2026-06-19 15:16:01 [INFO ] Epoch 188/300 | train=0.2539 dev=0.2487 |
lr=3.78e-04
2026-06-19 15:16:20 [INFO ] Epoch 189/300 | train=0.2542 dev=0.2518 |
lr=3.73e-04
2026-06-19 15:16:39 [INFO ] Epoch 190/300 | train=0.2544 dev=0.2492 |
lr=3.68e-04
2026-06-19 15:16:57 [INFO ] Epoch 191/300 | train=0.2534 dev=0.2483 |
lr=3.63e-04
2026-06-19 15:17:00 [INFO ] Best model saved (dev=0.2483)
2026-06-19 15:17:19 [INFO ] Epoch 192/300 | train=0.2545 dev=0.2489 |
lr=3.58e-04
2026-06-19 15:17:37 [INFO ] Epoch 193/300 | train=0.2535 dev=0.2480 |
lr=3.53e-04
2026-06-19 15:17:40 [INFO ] Best model saved (dev=0.2480)
2026-06-19 15:17:59 [INFO ] Epoch 194/300 | train=0.2541 dev=0.2475 |
lr=3.48e-04
2026-06-19 15:18:02 [INFO ] Best model saved (dev=0.2475)
2026-06-19 15:18:21 [INFO ] Epoch 195/300 | train=0.2527 dev=0.2477 |

```

```

lr=3.43e-04
2026-06-19 15:18:39 [INFO ] Epoch 196/300 | train=0.2530 dev=0.2488 |
lr=3.38e-04
2026-06-19 15:18:58 [INFO ] Epoch 197/300 | train=0.2535 dev=0.2482 |
lr=3.33e-04
2026-06-19 15:19:16 [INFO ] Epoch 198/300 | train=0.2532 dev=0.2473 |
lr=3.29e-04
2026-06-19 15:19:19 [INFO ] Best model saved (dev=0.2473)
2026-06-19 15:19:38 [INFO ] Epoch 199/300 | train=0.2518 dev=0.2475 |
lr=3.24e-04
2026-06-19 15:19:56 [INFO ] Epoch 200/300 | train=0.2521 dev=0.2473 |
lr=3.19e-04
2026-06-19 15:20:15 [INFO ] Epoch 201/300 | train=0.2521 dev=0.2489 |
lr=3.14e-04
2026-06-19 15:20:33 [INFO ] Epoch 202/300 | train=0.2518 dev=0.2468 |
lr=3.09e-04
2026-06-19 15:20:37 [INFO ] Best model saved (dev=0.2468)
2026-06-19 15:20:55 [INFO ] Epoch 203/300 | train=0.2512 dev=0.2464 |
lr=3.05e-04
2026-06-19 15:20:58 [INFO ] Best model saved (dev=0.2464)
2026-06-19 15:21:17 [INFO ] Epoch 204/300 | train=0.2529 dev=0.2468 |
lr=3.00e-04
2026-06-19 15:21:35 [INFO ] Epoch 205/300 | train=0.2515 dev=0.2460 |
lr=2.95e-04
2026-06-19 15:21:38 [INFO ] Best model saved (dev=0.2460)
2026-06-19 15:21:57 [INFO ] Epoch 206/300 | train=0.2520 dev=0.2466 |
lr=2.91e-04
2026-06-19 15:22:16 [INFO ] Epoch 207/300 | train=0.2514 dev=0.2472 |
lr=2.86e-04
2026-06-19 15:22:34 [INFO ] Epoch 208/300 | train=0.2511 dev=0.2462 |
lr=2.81e-04
2026-06-19 15:22:53 [INFO ] Epoch 209/300 | train=0.2504 dev=0.2461 |
lr=2.77e-04
2026-06-19 15:23:11 [INFO ] Epoch 210/300 | train=0.2506 dev=0.2457 |
lr=2.72e-04
2026-06-19 15:23:14 [INFO ] Best model saved (dev=0.2457)
2026-06-19 15:23:33 [INFO ] Epoch 211/300 | train=0.2515 dev=0.2492 |
lr=2.68e-04
2026-06-19 15:23:52 [INFO ] Epoch 212/300 | train=0.2510 dev=0.2460 |
lr=2.63e-04
2026-06-19 15:24:10 [INFO ] Epoch 213/300 | train=0.2501 dev=0.2453 |
lr=2.59e-04
2026-06-19 15:24:13 [INFO ] Best model saved (dev=0.2453)
2026-06-19 15:24:32 [INFO ] Epoch 214/300 | train=0.2501 dev=0.2460 |
lr=2.54e-04
2026-06-19 15:24:50 [INFO ] Epoch 215/300 | train=0.2500 dev=0.2457 |
lr=2.50e-04
2026-06-19 15:25:09 [INFO ] Epoch 216/300 | train=0.2502 dev=0.2453 |

```

```

lr=2.46e-04
2026-06-19 15:25:12 [INFO ] Best model saved (dev=0.2453)
2026-06-19 15:25:30 [INFO ] Epoch 217/300 | train=0.2498 dev=0.2452 |
lr=2.42e-04
2026-06-19 15:25:33 [INFO ] Best model saved (dev=0.2452)
2026-06-19 15:25:52 [INFO ] Epoch 218/300 | train=0.2496 dev=0.2460 |
lr=2.37e-04
2026-06-19 15:26:11 [INFO ] Epoch 219/300 | train=0.2501 dev=0.2455 |
lr=2.33e-04
2026-06-19 15:26:29 [INFO ] Epoch 220/300 | train=0.2496 dev=0.2449 |
lr=2.29e-04
2026-06-19 15:26:32 [INFO ] Best model saved (dev=0.2449)
2026-06-19 15:26:51 [INFO ] Epoch 221/300 | train=0.2489 dev=0.2448 |
lr=2.25e-04
2026-06-19 15:26:54 [INFO ] Best model saved (dev=0.2448)
2026-06-19 15:27:13 [INFO ] Epoch 222/300 | train=0.2488 dev=0.2457 |
lr=2.21e-04
2026-06-19 15:27:31 [INFO ] Epoch 223/300 | train=0.2490 dev=0.2445 |
lr=2.17e-04
2026-06-19 15:27:34 [INFO ] Best model saved (dev=0.2445)
2026-06-19 15:27:53 [INFO ] Epoch 224/300 | train=0.2488 dev=0.2446 |
lr=2.12e-04
2026-06-19 15:28:11 [INFO ] Epoch 225/300 | train=0.2490 dev=0.2444 |
lr=2.08e-04
2026-06-19 15:28:14 [INFO ] Best model saved (dev=0.2444)
2026-06-19 15:28:33 [INFO ] Epoch 226/300 | train=0.2488 dev=0.2443 |
lr=2.05e-04
2026-06-19 15:28:35 [INFO ] Best model saved (dev=0.2443)
2026-06-19 15:28:54 [INFO ] Epoch 227/300 | train=0.2490 dev=0.2440 |
lr=2.01e-04
2026-06-19 15:28:57 [INFO ] Best model saved (dev=0.2440)
2026-06-19 15:29:15 [INFO ] Epoch 228/300 | train=0.2486 dev=0.2441 |
lr=1.97e-04
2026-06-19 15:29:34 [INFO ] Epoch 229/300 | train=0.2491 dev=0.2442 |
lr=1.93e-04
2026-06-19 15:29:52 [INFO ] Epoch 230/300 | train=0.2482 dev=0.2446 |
lr=1.89e-04
2026-06-19 15:30:11 [INFO ] Epoch 231/300 | train=0.2482 dev=0.2444 |
lr=1.85e-04
2026-06-19 15:30:30 [INFO ] Epoch 232/300 | train=0.2479 dev=0.2437 |
lr=1.82e-04
2026-06-19 15:30:33 [INFO ] Best model saved (dev=0.2437)
2026-06-19 15:30:51 [INFO ] Epoch 233/300 | train=0.2487 dev=0.2437 |
lr=1.78e-04
2026-06-19 15:31:10 [INFO ] Epoch 234/300 | train=0.2476 dev=0.2435 |
lr=1.74e-04
2026-06-19 15:31:13 [INFO ] Best model saved (dev=0.2435)
2026-06-19 15:31:31 [INFO ] Epoch 235/300 | train=0.2486 dev=0.2445 |

```

```

lr=1.71e-04
2026-06-19 15:31:50 [INFO ] Epoch 236/300 | train=0.2481 dev=0.2435 |
lr=1.67e-04
2026-06-19 15:31:53 [INFO ] Best model saved (dev=0.2435)
2026-06-19 15:32:11 [INFO ] Epoch 237/300 | train=0.2475 dev=0.2440 |
lr=1.64e-04
2026-06-19 15:32:30 [INFO ] Epoch 238/300 | train=0.2474 dev=0.2431 |
lr=1.60e-04
2026-06-19 15:32:33 [INFO ] Best model saved (dev=0.2431)
2026-06-19 15:32:51 [INFO ] Epoch 239/300 | train=0.2476 dev=0.2437 |
lr=1.57e-04
2026-06-19 15:33:10 [INFO ] Epoch 240/300 | train=0.2472 dev=0.2435 |
lr=1.54e-04
2026-06-19 15:33:29 [INFO ] Epoch 241/300 | train=0.2470 dev=0.2434 |
lr=1.50e-04
2026-06-19 15:33:48 [INFO ] Epoch 242/300 | train=0.2470 dev=0.2429 |
lr=1.47e-04
2026-06-19 15:33:51 [INFO ] Best model saved (dev=0.2429)
2026-06-19 15:34:09 [INFO ] Epoch 243/300 | train=0.2477 dev=0.2434 |
lr=1.44e-04
2026-06-19 15:34:28 [INFO ] Epoch 244/300 | train=0.2469 dev=0.2435 |
lr=1.41e-04
2026-06-19 15:34:47 [INFO ] Epoch 245/300 | train=0.2466 dev=0.2451 |
lr=1.38e-04
2026-06-19 15:35:05 [INFO ] Epoch 246/300 | train=0.2467 dev=0.2436 |
lr=1.35e-04
2026-06-19 15:35:24 [INFO ] Epoch 247/300 | train=0.2467 dev=0.2433 |
lr=1.32e-04
2026-06-19 15:35:43 [INFO ] Epoch 248/300 | train=0.2464 dev=0.2442 |
lr=1.29e-04
2026-06-19 15:36:01 [INFO ] Epoch 249/300 | train=0.2465 dev=0.2431 |
lr=1.26e-04
2026-06-19 15:36:20 [INFO ] Epoch 250/300 | train=0.2461 dev=0.2434 |
lr=1.23e-04
2026-06-19 15:36:39 [INFO ] Epoch 251/300 | train=0.2462 dev=0.2427 |
lr=1.20e-04
2026-06-19 15:36:42 [INFO ] Best model saved (dev=0.2427)
2026-06-19 15:37:00 [INFO ] Epoch 252/300 | train=0.2461 dev=0.2467 |
lr=1.17e-04
2026-06-19 15:37:19 [INFO ] Epoch 253/300 | train=0.2465 dev=0.2424 |
lr=1.15e-04
2026-06-19 15:37:22 [INFO ] Best model saved (dev=0.2424)
2026-06-19 15:37:41 [INFO ] Epoch 254/300 | train=0.2460 dev=0.2431 |
lr=1.12e-04
2026-06-19 15:37:59 [INFO ] Epoch 255/300 | train=0.2463 dev=0.2424 |
lr=1.09e-04
2026-06-19 15:38:18 [INFO ] Epoch 256/300 | train=0.2460 dev=0.2421 |
lr=1.07e-04

```

```

2026-06-19 15:38:21 [INFO ] Best model saved (dev=0.2421)
2026-06-19 15:38:40 [INFO ] Epoch 257/300 | train=0.2457 dev=0.2453 |
lr=1.04e-04
2026-06-19 15:38:58 [INFO ] Epoch 258/300 | train=0.2462 dev=0.2424 |
lr=1.02e-04
2026-06-19 15:39:17 [INFO ] Epoch 259/300 | train=0.2456 dev=0.2418 |
lr=9.94e-05
2026-06-19 15:39:20 [INFO ] Best model saved (dev=0.2418)
2026-06-19 15:39:38 [INFO ] Epoch 260/300 | train=0.2457 dev=0.2423 |
lr=9.70e-05
2026-06-19 15:39:57 [INFO ] Epoch 261/300 | train=0.2455 dev=0.2419 |
lr=9.48e-05
2026-06-19 15:40:16 [INFO ] Epoch 262/300 | train=0.2454 dev=0.2431 |
lr=9.25e-05
2026-06-19 15:40:35 [INFO ] Epoch 263/300 | train=0.2458 dev=0.2417 |
lr=9.03e-05
2026-06-19 15:40:38 [INFO ] Best model saved (dev=0.2417)
2026-06-19 15:40:56 [INFO ] Epoch 264/300 | train=0.2453 dev=0.2417 |
lr=8.82e-05
2026-06-19 15:41:15 [INFO ] Epoch 265/300 | train=0.2453 dev=0.2419 |
lr=8.62e-05
2026-06-19 15:41:33 [INFO ] Epoch 266/300 | train=0.2453 dev=0.2418 |
lr=8.41e-05
2026-06-19 15:41:52 [INFO ] Epoch 267/300 | train=0.2456 dev=0.2417 |
lr=8.22e-05
2026-06-19 15:42:10 [INFO ] Epoch 268/300 | train=0.2451 dev=0.2419 |
lr=8.03e-05
2026-06-19 15:42:29 [INFO ] Epoch 269/300 | train=0.2450 dev=0.2413 |
lr=7.84e-05
2026-06-19 15:42:32 [INFO ] Best model saved (dev=0.2413)
2026-06-19 15:42:51 [INFO ] Epoch 270/300 | train=0.2450 dev=0.2415 |
lr=7.67e-05
2026-06-19 15:43:10 [INFO ] Epoch 271/300 | train=0.2452 dev=0.2415 |
lr=7.49e-05
2026-06-19 15:43:28 [INFO ] Epoch 272/300 | train=0.2449 dev=0.2414 |
lr=7.32e-05
2026-06-19 15:43:47 [INFO ] Epoch 273/300 | train=0.2450 dev=0.2413 |
lr=7.16e-05
2026-06-19 15:43:50 [INFO ] Best model saved (dev=0.2413)
2026-06-19 15:44:08 [INFO ] Epoch 274/300 | train=0.2447 dev=0.2425 |
lr=7.01e-05
2026-06-19 15:44:27 [INFO ] Epoch 275/300 | train=0.2447 dev=0.2414 |
lr=6.86e-05
2026-06-19 15:44:46 [INFO ] Epoch 276/300 | train=0.2447 dev=0.2412 |
lr=6.71e-05
2026-06-19 15:44:49 [INFO ] Best model saved (dev=0.2412)
2026-06-19 15:45:07 [INFO ] Epoch 277/300 | train=0.2451 dev=0.2415 |
lr=6.57e-05

```

```

2026-06-19 15:45:26 [INFO ] Epoch 278/300 | train=0.2446 dev=0.2428 |
lr=6.44e-05
2026-06-19 15:45:45 [INFO ] Epoch 279/300 | train=0.2446 dev=0.2410 |
lr=6.31e-05
2026-06-19 15:45:48 [INFO ] Best model saved (dev=0.2410)
2026-06-19 15:46:06 [INFO ] Epoch 280/300 | train=0.2446 dev=0.2425 |
lr=6.19e-05
2026-06-19 15:46:25 [INFO ] Epoch 281/300 | train=0.2444 dev=0.2425 |
lr=6.08e-05
2026-06-19 15:46:44 [INFO ] Epoch 282/300 | train=0.2445 dev=0.2413 |
lr=5.97e-05
2026-06-19 15:47:02 [INFO ] Epoch 283/300 | train=0.2444 dev=0.2424 |
lr=5.86e-05
2026-06-19 15:47:21 [INFO ] Epoch 284/300 | train=0.2445 dev=0.2422 |
lr=5.76e-05
2026-06-19 15:47:40 [INFO ] Epoch 285/300 | train=0.2443 dev=0.2411 |
lr=5.67e-05
2026-06-19 15:47:58 [INFO ] Epoch 286/300 | train=0.2444 dev=0.2408 |
lr=5.58e-05
2026-06-19 15:48:01 [INFO ] Best model saved (dev=0.2408)
2026-06-19 15:48:20 [INFO ] Epoch 287/300 | train=0.2444 dev=0.2409 |
lr=5.50e-05
2026-06-19 15:48:39 [INFO ] Epoch 288/300 | train=0.2443 dev=0.2415 |
lr=5.43e-05
2026-06-19 15:48:57 [INFO ] Epoch 289/300 | train=0.2443 dev=0.2414 |
lr=5.36e-05
2026-06-19 15:49:16 [INFO ] Epoch 290/300 | train=0.2441 dev=0.2414 |
lr=5.30e-05
2026-06-19 15:49:35 [INFO ] Epoch 291/300 | train=0.2445 dev=0.2410 |
lr=5.24e-05
2026-06-19 15:49:53 [INFO ] Epoch 292/300 | train=0.2440 dev=0.2409 |
lr=5.19e-05
2026-06-19 15:50:12 [INFO ] Epoch 293/300 | train=0.2440 dev=0.2423 |
lr=5.15e-05
2026-06-19 15:50:31 [INFO ] Epoch 294/300 | train=0.2442 dev=0.2414 |
lr=5.11e-05
2026-06-19 15:50:49 [INFO ] Epoch 295/300 | train=0.2440 dev=0.2415 |
lr=5.07e-05
2026-06-19 15:51:08 [INFO ] Epoch 296/300 | train=0.2442 dev=0.2408 |
lr=5.05e-05
2026-06-19 15:51:11 [INFO ] Best model saved (dev=0.2408)
2026-06-19 15:51:29 [INFO ] Epoch 297/300 | train=0.2439 dev=0.2415 |
lr=5.03e-05
2026-06-19 15:51:48 [INFO ] Epoch 298/300 | train=0.2439 dev=0.2433 |
lr=5.01e-05
2026-06-19 15:52:07 [INFO ] Epoch 299/300 | train=0.2438 dev=0.2408 |
lr=5.00e-05
2026-06-19 15:52:10 [INFO ] Best model saved (dev=0.2408)

```

```
2026-06-19 15:52:28 [INFO ] Epoch 300/300 | train=0.2440 dev=0.2417 |
lr=5.00e-05
2026-06-19 15:52:29 [INFO ] Training done. Best dev: 0.2408
```

```
real    98m23.737s
user    95m15.133s
sys     3m42.999s
```

```
[36]: !python ./myanmar_tts_transformer.py --help
```

```
usage: myanmar_tts_transformer.py [-h] [--stage {prep,train,synth,eval,all}]
                                   [--check_deps] [--data_dir DATA_DIR]
                                   [--tsv_file TSV_FILE]
                                   [--output_dir OUTPUT_DIR]
                                   [--train_ratio TRAIN_RATIO]
                                   [--dev_ratio DEV_RATIO]
                                   [--max_samples MAX_SAMPLES]
                                   [--sample_rate SAMPLE_RATE] [--n_fft N_FFT]
                                   [--hop_length HOP_LENGTH]
                                   [--win_length WIN_LENGTH] [--n_mels N_MELS]
                                   [--fmin FMIN] [--fmax FMAX]
                                   [--d_model D_MODEL] [--n_heads N_HEADS]
                                   [--n_layers N_LAYERS] [--ffn_dim FFN_DIM]
                                   [--dropout DROPOUT] [--epochs EPOCHS]
                                   [--warmup_epochs WARMUP_EPOCHS] [--lr LR]
                                   [--batch_size BATCH_SIZE]
                                   [--weight_decay WEIGHT_DECAY]
                                   [--grad_clip GRAD_CLIP]
                                   [--num_workers NUM_WORKERS]
                                   [--save_every SAVE_EVERY]
                                   [--log_steps LOG_STEPS]
                                   [--dur_loss_weight DUR_LOSS_WEIGHT]
                                   [--synth_text SYNTH_TEXT]
                                   [--synth_out SYNTH_OUT]
                                   [--speed_factor SPEED_FACTOR]
                                   [--eval_n EVAL_N] [--log_file LOG_FILE]
                                   [--verbose] [--seed SEED] [--resume RESUME]
                                   [--list_samples]
                                   [--gt_sample_id GT_SAMPLE_ID] [--use_gt_mel]
                                   [--gt_compare] [--no_neural_vocoder]
```

Myanmar TTS Demo v4 - TransformerTTS (autoregressive)

options:

```
-h, --help          show this help message and exit
--stage {prep,train,synth,eval,all}
--check_deps
--data_dir DATA_DIR
```



```

--tsv_file TSV_FILE
--output_dir OUTPUT_DIR
--train_ratio TRAIN_RATIO
--dev_ratio DEV_RATIO
--max_samples MAX_SAMPLES
--sample_rate SAMPLE_RATE
--n_fft N_FFT
--hop_length HOP_LENGTH
--win_length WIN_LENGTH
--n_mels N_MELS
--fmin FMIN
--fmax FMAX
--d_model D_MODEL
--n_heads N_HEADS
--n_layers N_LAYERS    Encoder AND decoder layers
--ffn_dim FFN_DIM
--dropout DROPOUT
--epochs EPOCHS
--warmup_epochs WARMUP_EPOCHS
--lr LR
--batch_size BATCH_SIZE
--weight_decay WEIGHT_DECAY
--grad_clip GRAD_CLIP
--num_workers NUM_WORKERS
--save_every SAVE_EVERY
--log_steps LOG_STEPS
--dur_loss_weight DUR_LOSS_WEIGHT
                                [DEPRECATED] ignored in v4 (no duration predictor)
--synth_text SYNTH_TEXT
--synth_out SYNTH_OUT
--speed_factor SPEED_FACTOR
                                0.8=slower 1.0=normal 1.2=faster
--eval_n EVAL_N
--log_file LOG_FILE
--verbose
--seed SEED
--resume RESUME
--list_samples
--gt_sample_id GT_SAMPLE_ID
--use_gt_mel
--gt_compare
--no_neural_vocoder

```

4.3 List Samples

```
[37]: !python myanmar_tts_transformer.py --stage synth --output_dir ./tts_output
      ↪--list_samples
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
2026-06-20 13:54:18 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 13:54:18 [INFO    ] Sample IDs (first 20):
2026-06-20 13:54:18 [INFO    ] bur_8266_7413365509                ဥပမာ
မိုးဟေကို ဆိုရင် ဆောင်းဟေကို နာမည် ကို ဆင်တူ
2026-06-20 13:54:18 [INFO    ] bur_7865_2573585785                ဒါလေမဲ့ ဒီ
ကျောက် က ရာသက်ပန် မကုန်အောင် ထွက်နေမှာ
2026-06-20 13:54:18 [INFO    ] bur_5189_3215659791                ကိုကျော်သူ
မမေသန်းနု ဒေါ်အေးအေးသင်းတို့ ပါတယ်
2026-06-20 13:54:18 [INFO    ] bur_0644_5980380162                အမှန်တော့
မှတ်တမ်း ကို ဖလင် နဲ့ ရိုက် ပြီး ပြတဲ့ ရ
2026-06-20 13:54:18 [INFO    ] bur_5932_9903725735                ကျနော် က
ကျော်ထွဋ်ဆွေတို့ ကျောက်ဂျက်တို့ နဲ့တော့ သ
2026-06-20 13:54:18 [INFO    ] bur_5932_6467856225                ရုပ်ရှင်ကားကြီးတွေ ရိုက်ရင် ဖလင် က တအား ကုန်တယ်
2026-06-20 13:54:18 [INFO    ] bur_7543_1589569333                ကာတွန်း
ရိုက်အောင် ကို ပုသိမ်မြို့ မှာ အဖ ဦးမောင်က
2026-06-20 13:54:18 [INFO    ] bur_9762_6350280557                ကြည့်ချင်တယ် ဆိုရင်လည်း ခရီးစဉ် တခါတည်း ထည့် ဆွဲထား
2026-06-20 13:54:18 [INFO    ] bur_5903_4827254748                အခု က ဖုန်း
တစ်လုံး ပဲ ထားရတော့တယ် ဟု ဆိုသည်
2026-06-20 13:54:18 [INFO    ] bur_3260_5647990176                ကိုယ် ကလည်း
အချိန် မပေးနိုင်ဘူး ဆိုတော့ ဆရာဝန်ကြီး
2026-06-20 13:54:18 [INFO    ] bur_5362_4730425798                ခေတ်မှီချင်သူတွေ ကတော့ ရေခဲမုန့် ကျွေးကြတာပေါ့
2026-06-20 13:54:18 [INFO    ] bur_8266_3739445651                ကဲ အခု တော့
ဘယ်နှယ့် ရှိစ
2026-06-20 13:54:18 [INFO    ] bur_9135_9365873650                အထက်
မြန်မာပြည် မန္တလေး မြို့ ခွဲလည်း ဘတ်စကတ်ဘော ကလပ်
2026-06-20 13:54:18 [INFO    ] bur_5932_0137380639                အရင်
တုန်းကတော့ ဒီ ကိစ္စ ကို အားလုံး လေးငါးခြောက်ယ
```

2026-06-20 13:54:18 [INFO]	bur_8698_8976831996	မင်း အမေ
ကို ကောင်းကောင်း ဆော်သွား လို့ မင်း ထွက်လ		
2026-06-20 13:54:18 [INFO]	bur_0366_2701213104	ကမ္ဘာအနံ့
မှ သတင်း အချက်အလက်တွေ ကို သိနိုင်သည်		
2026-06-20 13:54:18 [INFO]	bur_0644_2661360424	ဆိုနီ ကရော
ဘယ်လိုတွေ ချပြမလဲ လေ့လာကြည့်ရအောင်		
2026-06-20 13:54:18 [INFO]	bur_7712_5029179191	ကမ္ဘာဇ
ဘောလုံး အသင်း က ဥက္ကဋ္ဌ ဦးခင်မောင်လတ် က အရင်		
2026-06-20 13:54:18 [INFO]	bur_5362_4875790905	ကျွန်တော်
ကလည်း ဒိတ်ဒိတ်ကျ အောင်မြင်နေတဲ့ မင်းသား		
2026-06-20 13:54:18 [INFO]	bur_3260_2712519099	ကျနော် က
ရိက္ခာ ယူခိုင်း တုန်း ထွက်ပြေးလာတာပါ		

4.4 Vocoder Check

```
[38]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_5362_4875790905
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 13:56:56 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 13:56:56 [INFO ] Found bur_5362_4875790905 in train
2026-06-20 13:56:56 [INFO ] GT text: ကျွန်တော် ကလည်း ဒိတ်ဒိတ်ကျ
အောင်မြင်နေတဲ့ မင်းသား မှ မဟုတ်တာလေ
2026-06-20 13:56:56 [INFO ] GT mel: min=-80.0 max=0.0 mean=-57.5 dB
shape=(522, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:809: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 13:56:59 [INFO ] Mel plot → gt_bur_5362_4875790905.png
2026-06-20 13:56:59 [INFO ] GT audio → gt_bur_5362_4875790905.wav (6.05s)

real    0m4.488s
user    0m15.123s
sys     0m0.439s
```

```
[39]: import IPython.display as ipd

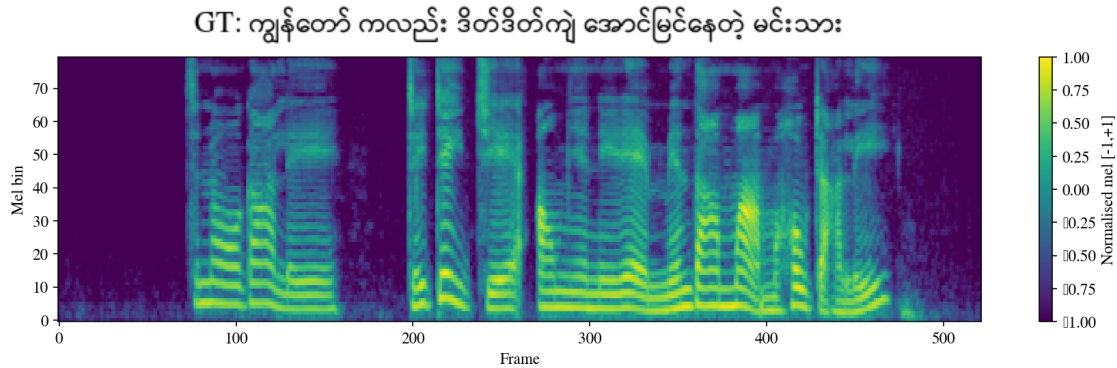
# For a local file
```

```
ipd.Audio("./gt_bur_5362_4875790905.wav")
```

[39]: <IPython.lib.display.Audio object>

```
[31]: from IPython.display import Image
Image(filename='gt_bur_5362_4875790905.png')
```

[31]:



4.5 Full TTS

```
[40]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output \
--synth_text "ကျွန်တော် ကလည်း ဒီတဒီတကျ အောင်မြင်နေတဲ့ မင်းသား မှ မဟုတ်တာလေ" \
--gt_compare
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
```

```
2026-06-20 13:59:10 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
```

```
2026-06-20 13:59:11 [INFO ] Loaded TransformerTTS: epoch=299 dev=0.2408
```

```
2026-06-20 13:59:11 [INFO ] Synthesizing: 'ကျွန်တော် ကလည်း ဒီတဒီတကျ
အောင်မြင်နေတဲ့ မင်းသား မှ မဟုတ်တာလေ'
```

```
2026-06-20 13:59:11 [INFO ] length : 65 tokens × 7.3 fpc 472 frames
(floor=236, cap=1200)
```

```
2026-06-20 13:59:12 [INFO ] mel_norm : min=-1.035 max=-0.460
mean=-0.998
```

```
2026-06-20 13:59:12 [INFO ] mel_db : min=-80.0 max=-58.4 mean=-79.6
dB
```

```

2026-06-20 13:59:12 [INFO    ] shape      : (237, 80) (2.75s at default
hop/sr)
2026-06-20 13:59:12 [WARNING ] mel_db max=-58.4 dB. Model may need more
epochs (try 200+).
2026-06-20 13:59:12 [INFO    ] Acoustic model: 989 ms
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:809: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
    plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 13:59:12 [INFO    ] Mel plot → tts_output/synthesized.png
2026-06-20 13:59:14 [INFO    ] Vocoder: 1608 ms | Audio →
tts_output/synthesized.wav (2.74s)
2026-06-20 13:59:14 [INFO    ] Found text match in train
2026-06-20 13:59:16 [INFO    ] GT comparison → synthesized_GT.wav
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:824: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
    plt.tight_layout(); plt.savefig(save_path,dpi=150); plt.close(fig)
2026-06-20 13:59:16 [INFO    ] Comparison plot → synthesized_compare.png

real    0m7.147s
user    0m21.415s
sys      0m0.664s

```

```

[41]: import IPython.display as ipd

      # For a local file
      ipd.Audio("./tts_output/synthesized.wav")

```

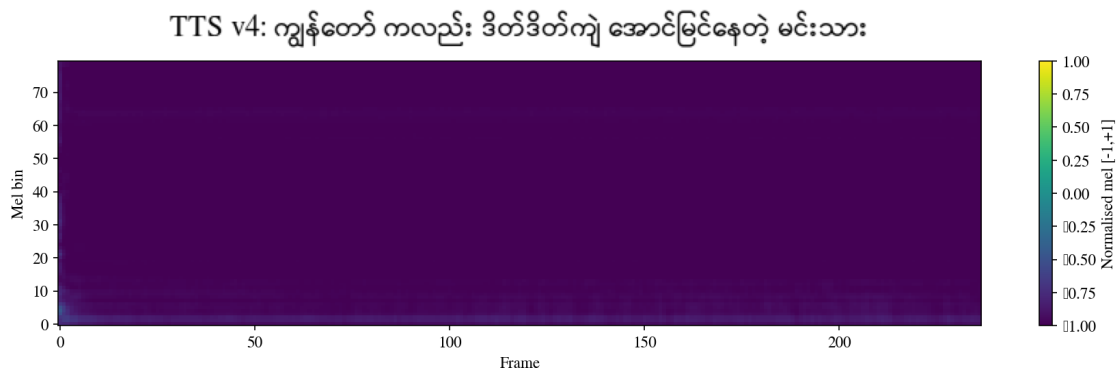
[41]: <IPython.lib.display.Audio object>

```

[42]: from IPython.display import Image
      Image(filename='./tts_output/synthesized.png')

```

[42]:

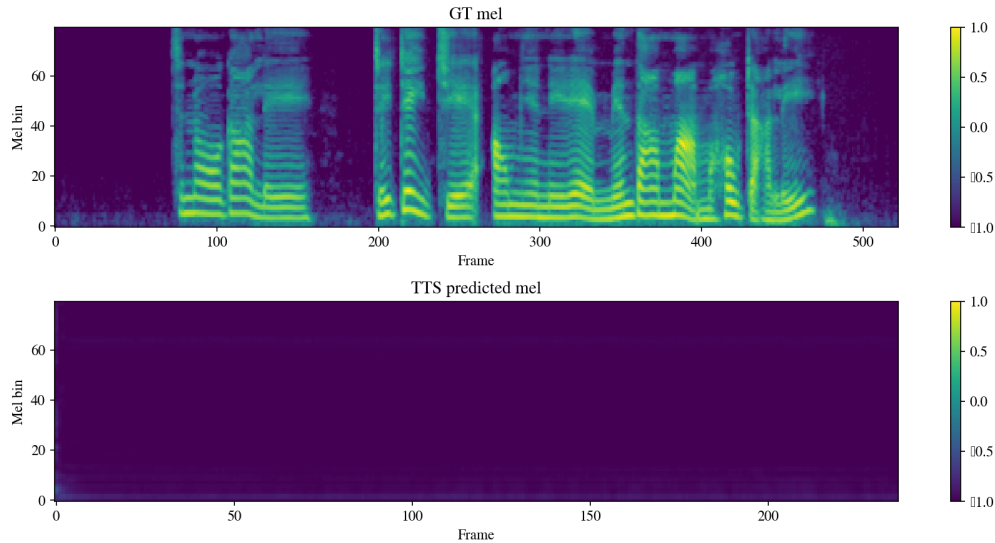


```

[43]: from IPython.display import Image
      Image(filename='synthesized_compare.png')

```

[43]:



```
[44]: !time python myanmar_tts_transformer.py --stage synth \
      --output_dir ./tts_output --synth_text "မင်္ဂလာပါ" --gt_compare
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 14:01:49 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 14:01:49 [INFO    ] Loaded TransformerTTS: epoch=299 dev=0.2408
2026-06-20 14:01:49 [INFO    ] Synthesizing: 'မင်္ဂလာပါ'
2026-06-20 14:01:49 [INFO    ] length    : 11 tokens × 7.3 fpc  80 frames
(floor=40, cap=220)
2026-06-20 14:01:50 [INFO    ] mel_norm  : min=-1.040 max=-0.464
mean=-0.990
2026-06-20 14:01:50 [INFO    ] mel_db    : min=-80.0 max=-58.5 mean=-79.4
dB
2026-06-20 14:01:50 [INFO    ] shape     : (41, 80) (0.48s at default
hop/sr)
2026-06-20 14:01:50 [WARNING ] mel_db max=-58.5 dB. Model may need more
epochs (try 200+).
2026-06-20 14:01:50 [INFO    ] Acoustic model: 390 ms
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:809: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
```

```
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 14:01:50 [INFO ] Mel plot → tts_output/synthesized.png
2026-06-20 14:01:51 [INFO ] Vocoder: 988 ms | Audio →
tts_output/synthesized.wav (0.46s)
2026-06-20 14:01:51 [INFO ] No exact GT match found.

real    0m3.747s
user    0m11.201s
sys     0m0.691s
```

```
[45]: import IPython.display as ipd

# For a local file
ipd.Audio("./tts_output/synthesized.wav")
```

```
[45]: <IPython.lib.display.Audio object>
```

4.6 Vocoder Check-2

လက်ရှိ Vocoder ရဲ့ resynthesized အသံတွေကို အသေးစိတ် လေ့လာနိုင်အောင် ထပ်ဖြည့် run ပြထားတာပါ။
Vocoder ရဲ့ speed ကိုလည်း ကြည့်ကြရအောင်။

```
[50]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_8266_7413365509
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 19:21:24 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:21:24 [INFO ] Found bur_8266_7413365509 in train
2026-06-20 19:21:24 [INFO ] GT text: ဥပမာ မိုးဟေကို ဆိုရင် ဆောင်းဟေကို
နာမည် ကို ဆင်တူ လေးထားတာမျိုး ပေါ့
2026-06-20 19:21:24 [INFO ] GT mel: min=-80.0 max=0.0 mean=-61.6 dB
shape=(530, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:21:26 [INFO ] Mel plot → gt_bur_8266_7413365509.png
2026-06-20 19:21:26 [INFO ] GT audio → gt_bur_8266_7413365509.wav (6.14s)

real    0m4.454s
user    0m15.203s
```

sys 0m0.434s

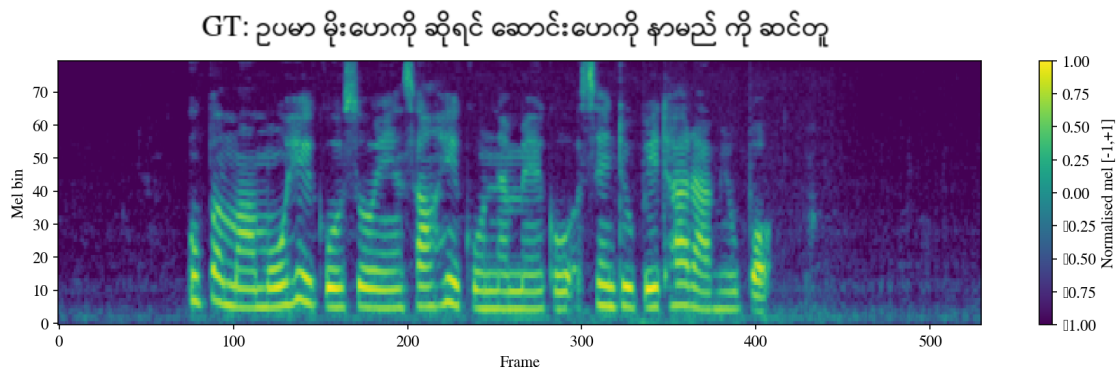
```
[53]: import IPython.display as ipd

# For a local file
ipd.Audio("./gt_bur_8266_7413365509.wav")
```

[53]: <IPython.lib.display.Audio object>

```
[54]: from IPython.display import Image
Image(filename='gt_bur_8266_7413365509.png')
```

[54]:



```
[55]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_7865_2573585785
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 19:29:27 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:29:27 [INFO    ] Found bur_7865_2573585785 in train
2026-06-20 19:29:27 [INFO    ] GT text: ဒါပေမဲ့ ဒီ ကျောက် က ရာသက်ပန်
မကုန်အောင် ထွက်နေမှာ မဟုတ်ဘူး
2026-06-20 19:29:27 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-66.1 dB
shape=(596, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:29:30 [INFO    ] Mel plot → gt_bur_7865_2573585785.png
```


2026-06-20 19:29:30 [INFO] GT audio → gt_bur_7865_2573585785.wav (6.91s)

```
real    0m4.762s
user    0m16.203s
sys     0m0.409s
```

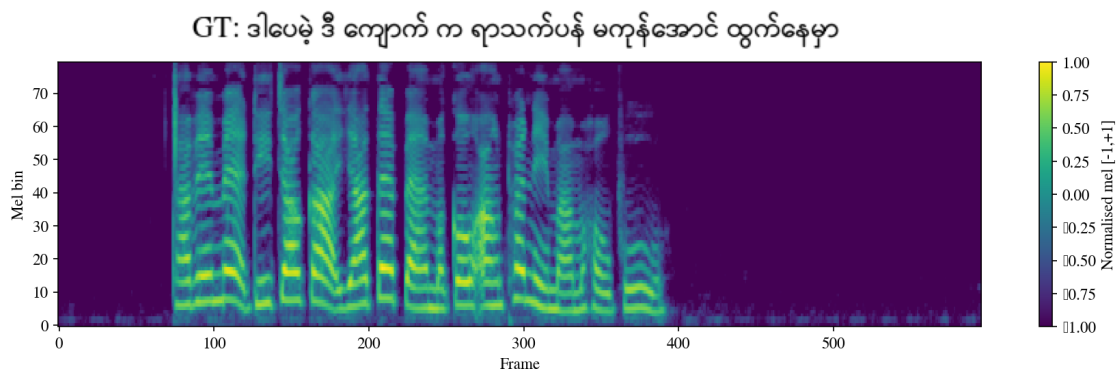
```
[59]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_7865_2573585785.wav')
```

[59]: <IPython.lib.display.Audio object>

```
[61]: from IPython.display import Image
Image(filename='gt_bur_7865_2573585785.png')
```

[61]:



```
[62]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_5189_3215659791
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
2026-06-20 19:33:03 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:33:03 [INFO ] Found bur_5189_3215659791 in train
2026-06-20 19:33:03 [INFO ] GT text: ကိုကျော်သူ မမေသန်းနု
ဒေါ်အေးအေးသင်းတို့ ပါတယ်
2026-06-20 19:33:03 [INFO ] GT mel: min=-80.0 max=0.0 mean=-67.1 dB
shape=(456, 80)
```

```
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:33:06 [INFO ] Mel plot → gt_bur_5189_3215659791.png
2026-06-20 19:33:06 [INFO ] GT audio → gt_bur_5189_3215659791.wav (5.28s)
```

```
real    0m4.240s
user    0m13.636s
sys     0m0.436s
```

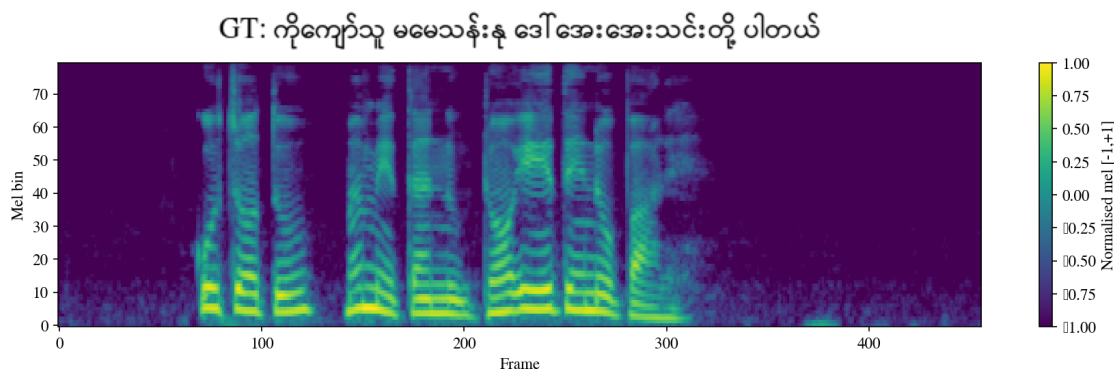
```
[63]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_5189_3215659791.wav')
```

```
[63]: <IPython.lib.display.Audio object>
```

```
[64]: from IPython.display import Image
Image(filename='gt_bur_5189_3215659791.png')
```

```
[64]:
```



```
[65]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_0644_5980380162
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 19:34:31 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:34:31 [INFO ] Found bur_0644_5980380162 in train
```

```

2026-06-20 19:34:31 [INFO    ] GT text: အမှန်တော့ မှတ်တမ်း ကို ဖလင် နဲ့ ရိုက်
ပြီး ပြတဲ့ ရုပ်ရှင် ပါ
2026-06-20 19:34:31 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-58.1 dB
shape=(434, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:34:34 [INFO    ] Mel plot → gt_bur_0644_5980380162.png
2026-06-20 19:34:34 [INFO    ] GT audio → gt_bur_0644_5980380162.wav (5.03s)

real    0m4.116s
user    0m13.282s
sys      0m0.406s

```

```

[66]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_0644_5980380162.wav')

```

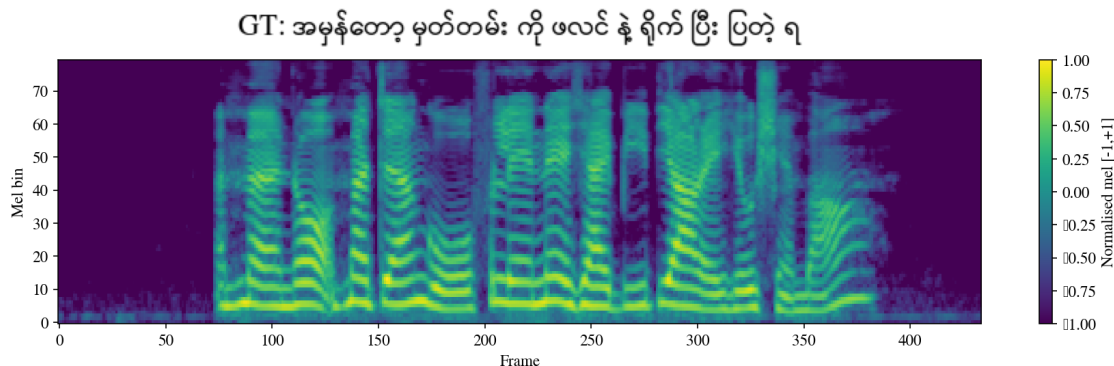
[66]: <IPython.lib.display.Audio object>

```

[67]: from IPython.display import Image
Image(filename='gt_bur_0644_5980380162.png')

```

[67]:



```

[68]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_5932_9903725735

```

```

/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classific
ation/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classific
ation/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available

```

```
warnings.warn(
2026-06-20 19:36:09 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:36:09 [INFO    ] Found bur_5932_9903725735 in train
2026-06-20 19:36:09 [INFO    ] GT text: ကျနော် က ကျော်ထွဋ်ဆွေတို့
ကျောက်ဂျက်တို့ နဲ့တော့ သီချင်းတွေ ဆိုခဲ့တာ အောင်မြင်တယ်
2026-06-20 19:36:09 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-54.5 dB
shape=(419, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:36:12 [INFO    ] Mel plot → gt_bur_5932_9903725735.png
2026-06-20 19:36:12 [INFO    ] GT audio → gt_bur_5932_9903725735.wav (4.85s)

real    0m4.128s
user    0m12.883s
sys      0m0.444s
```

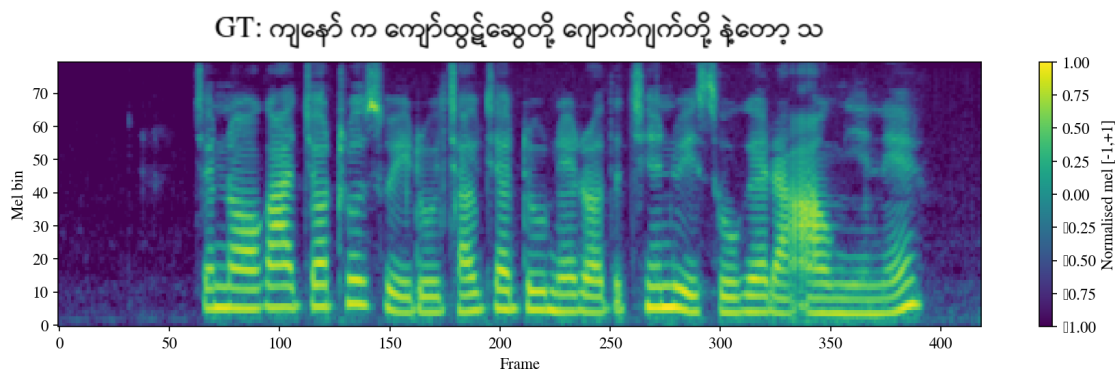
```
[69]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_5932_9903725735.wav')
```

```
[69]: <IPython.lib.display.Audio object>
```

```
[70]: from IPython.display import Image
Image(filename='gt_bur_5932_9903725735.png')
```

```
[70]:
```



```
[71]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_5932_6467856225
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classific
ation/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
```

```
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classific
ation/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 19:37:43 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:37:43 [INFO    ] Found bur_5932_6467856225 in train
2026-06-20 19:37:43 [INFO    ] GT text: ရုပ်ရှင်ကားကြီးတွေ ရိုက်ရင် ဖလင် က
တအား ကုန်တယ်
2026-06-20 19:37:43 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-52.6 dB
shape=(317, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:37:45 [INFO    ] Mel plot → gt_bur_5932_6467856225.png
2026-06-20 19:37:45 [INFO    ] GT audio → gt_bur_5932_6467856225.wav (3.67s)

real    0m3.868s
user    0m13.804s
sys     0m0.416s
```

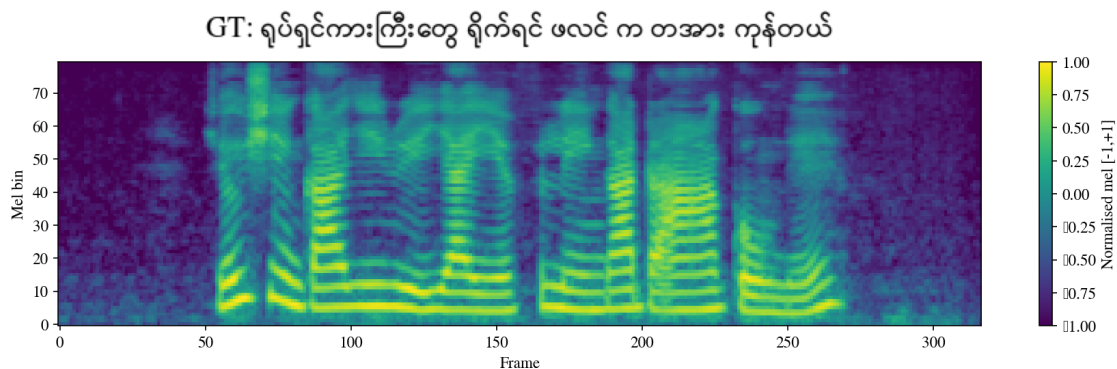
```
[72]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_5932_6467856225.wav')
```

[72]: <IPython.lib.display.Audio object>

```
[73]: from IPython.display import Image
Image(filename='gt_bur_5932_6467856225.png')
```

[73]:



```
[74]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_7543_1589569333
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
2026-06-20 19:38:58 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:38:58 [INFO    ] Found bur_7543_1589569333 in train
2026-06-20 19:38:58 [INFO    ] GT text: ကာတွန်း ရှိန်အောင် ကို ပုသိမ်မြို့ မှာ
အဖ ဦးမောင်ကွန်း နဲ့ အမိ ဒေါ်သန်းရင်တို့ က မွေးဖွားတယ်
2026-06-20 19:38:58 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-59.3 dB
shape=(522, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
  plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:39:00 [INFO    ] Mel plot → gt_bur_7543_1589569333.png
2026-06-20 19:39:00 [INFO    ] GT audio → gt_bur_7543_1589569333.wav (6.05s)

real    0m4.421s
user    0m15.160s
sys     0m0.402s
```

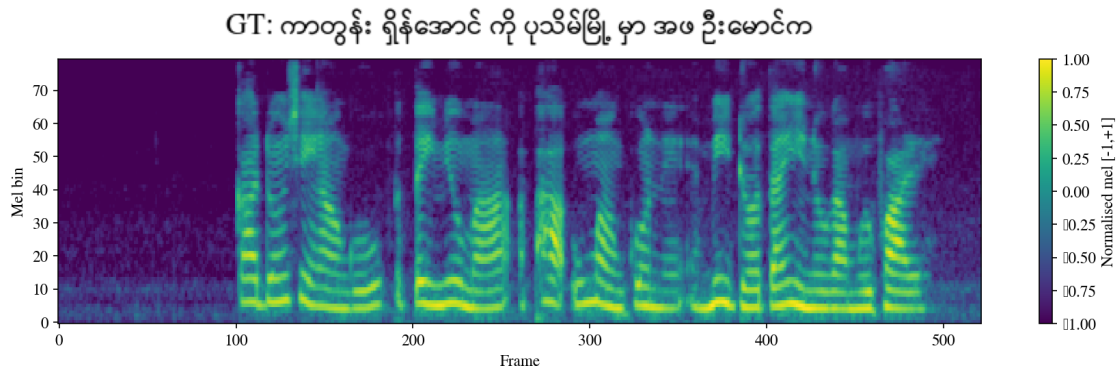
```
[75]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_7543_1589569333.wav')
```

```
[75]: <IPython.lib.display.Audio object>
```

```
[76]: from IPython.display import Image
Image(filename='gt_bur_7543_1589569333.png')
```

```
[76]:
```



```
[77]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_9762_6350280557
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classific
ation/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classific
ation/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 19:40:18 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:40:18 [INFO    ] Found bur_9762_6350280557 in train
2026-06-20 19:40:18 [INFO    ] GT text: ကြည့်ချင်တယ် ဆိုရင်လည်း ခရီးစဉ်
တခါတည်း ထည့် ဆွဲထားနော
2026-06-20 19:40:18 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-60.4 dB
shape=(434, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:40:21 [INFO    ] Mel plot → gt_bur_9762_6350280557.png
2026-06-20 19:40:21 [INFO    ] GT audio → gt_bur_9762_6350280557.wav (5.03s)

real    0m4.176s
user    0m13.250s
sys     0m0.438s
```

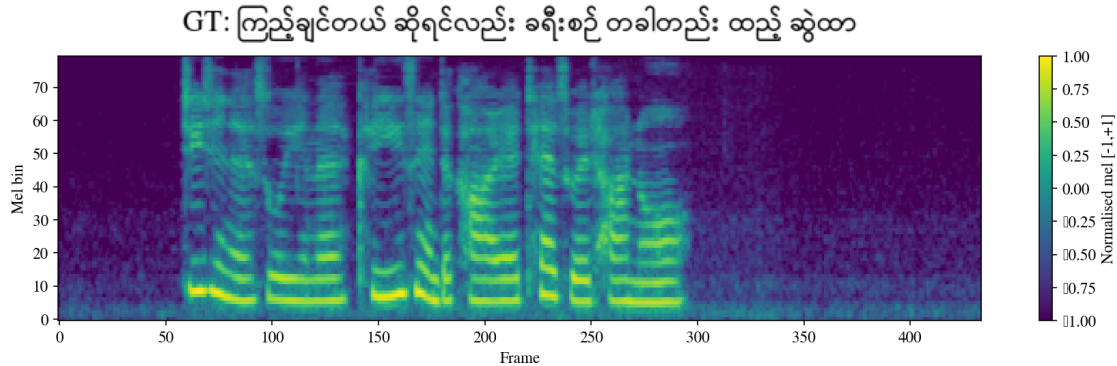
```
[78]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_9762_6350280557.wav')
```


[78]: <IPython.lib.display.Audio object>

```
[79]: from IPython.display import Image
Image(filename='gt_bur_9762_6350280557.png')
```

[79]:



```
[80]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_5903_4827254748
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 19:41:50 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:41:50 [INFO    ] Found bur_5903_4827254748 in train
2026-06-20 19:41:50 [INFO    ] GT text: အခု က ဖုန်း တစ်လုံး ပဲ ထားရတော့တယ် ဟု
ဆိုသည်
2026-06-20 19:41:50 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-54.1 dB
shape=(295, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:41:52 [INFO    ] Mel plot → gt_bur_5903_4827254748.png
2026-06-20 19:41:52 [INFO    ] GT audio → gt_bur_5903_4827254748.wav (3.41s)

real    0m3.660s
user    0m11.998s
sys     0m0.427s
```



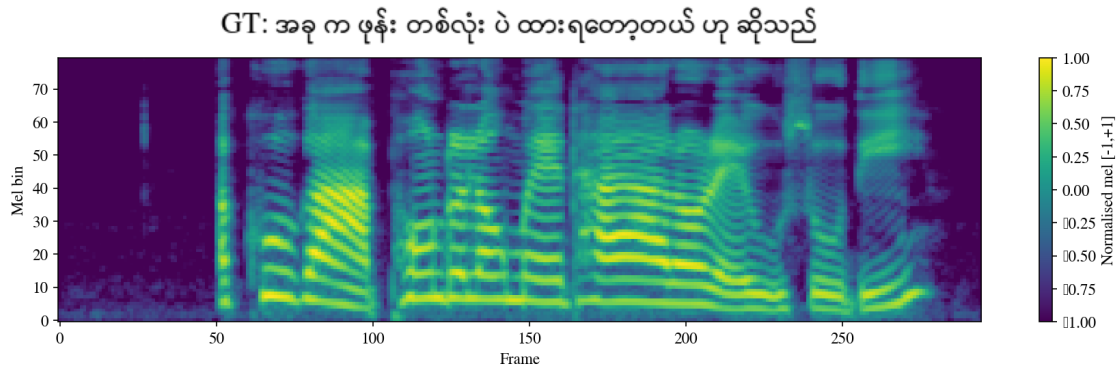
```
[81]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_5903_4827254748.wav')
```

[81]: <IPython.lib.display.Audio object>

```
[85]: from IPython.display import Image
Image(filename='./gt_bur_5903_4827254748.png')
```

[85]:



```
[86]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_3260_5647990176
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 19:43:40 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:43:40 [INFO    ] Found bur_3260_5647990176 in train
2026-06-20 19:43:40 [INFO    ] GT text: ကိုယ် ကလည်း အချိန် မပေးနိုင်ဘူး
ဆိုတော့ ဆရာဝန်ကြီးတွေ နဲ့ တိုင်ပင်ရတယ်
2026-06-20 19:43:40 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-54.5 dB
shape=(390, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:43:42 [INFO    ] Mel plot → gt_bur_3260_5647990176.png
2026-06-20 19:43:42 [INFO    ] GT audio → gt_bur_3260_5647990176.wav (4.52s)
```

```
real    0m3.976s
user    0m12.725s
sys     0m0.403s
```

```
[87]: import IPython.display as ipd

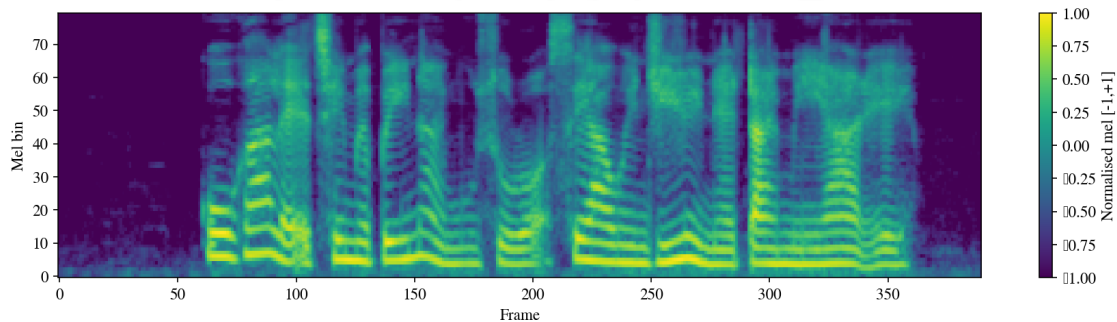
# For a local file
ipd.Audio('./gt_bur_3260_5647990176.wav')
```

```
[87]: <IPython.lib.display.Audio object>
```

```
[88]: from IPython.display import Image
Image(filename='gt_bur_3260_5647990176.png')
```

```
[88]:
```

GT: ကိုယ် ကလည်း အချိန် မပေးနိုင်ဘူး ဆိုတော့ ဆရာဝန်ကြီး



```
[89]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_5362_4730425798
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 19:45:12 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:45:12 [INFO    ] Found bur_5362_4730425798 in train
2026-06-20 19:45:12 [INFO    ] GT text: ခေတ်မှီချင်သူတွေ ကတော့ ရေခဲမုန့်
ကျွေးကြတာပေါ့
2026-06-20 19:45:12 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-61.8 dB
shape=(456, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
```

Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.

```
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:45:14 [INFO ] Mel plot → gt_bur_5362_4730425798.png
2026-06-20 19:45:14 [INFO ] GT audio → gt_bur_5362_4730425798.wav (5.28s)
```

```
real    0m4.163s
user    0m14.402s
sys     0m0.469s
```

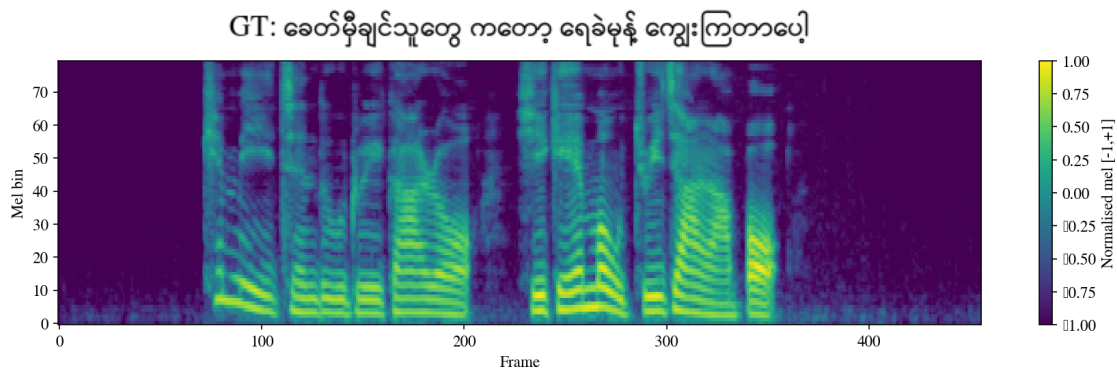
```
[90]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_5362_4730425798.wav')
```

[90]: <IPython.lib.display.Audio object>

```
[91]: from IPython.display import Image
Image(filename='gt_bur_5362_4730425798.png')
```

[91]:



```
[92]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_8266_3739445651
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
2026-06-20 19:46:56 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:46:56 [INFO ] Found bur_8266_3739445651 in train
2026-06-20 19:46:56 [INFO ] GT text: ကဲ အခု တော့ ဘယ်နှယ့် ရှိစ
```

```

2026-06-20 19:46:56 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-65.5 dB
shape=(361, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
  plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:46:58 [INFO    ] Mel plot → gt_bur_8266_3739445651.png
2026-06-20 19:46:58 [INFO    ] GT audio → gt_bur_8266_3739445651.wav (4.18s)

```

```

real    0m3.867s
user    0m12.524s
sys     0m0.423s

```

```

[93]: import IPython.display as ipd

      # For a local file
      ipd.Audio('./gt_bur_8266_3739445651.wav')

```

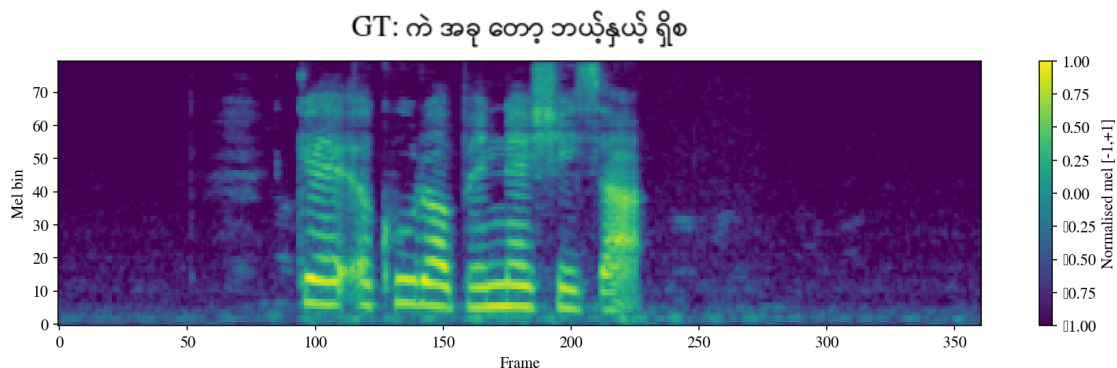
```
[93]: <IPython.lib.display.Audio object>
```

```

[94]: from IPython.display import Image
      Image(filename='gt_bur_8266_3739445651.png')

```

```
[94]:
```



```

[95]: !time python myanmar_tts_transformer.py --stage synth \
      --output_dir ./tts_output --gt_sample_id bur_9135_9365873650

```

```

/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
2026-06-20 19:48:09 [WARNING ] WaveGlow unavailable (RuntimeError), using

```

```

Griffin-Lim.
2026-06-20 19:48:09 [INFO ] Found bur_9135_9365873650 in train
2026-06-20 19:48:09 [INFO ] GT text: အထက် မြန်မာပြည် မန္တလေး မြို့ ခြံလည်း
ဘတ်စကတ်ဘော ကလပ်အသင်းများ ရှိသည်
2026-06-20 19:48:09 [INFO ] GT mel: min=-80.0 max=0.0 mean=-62.8 dB
shape=(368, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:48:12 [INFO ] Mel plot → gt_bur_9135_9365873650.png
2026-06-20 19:48:12 [INFO ] GT audio → gt_bur_9135_9365873650.wav (4.26s)

real    0m3.869s
user    0m12.548s
sys     0m0.412s

```

```

[96]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_9135_9365873650.wav')

```

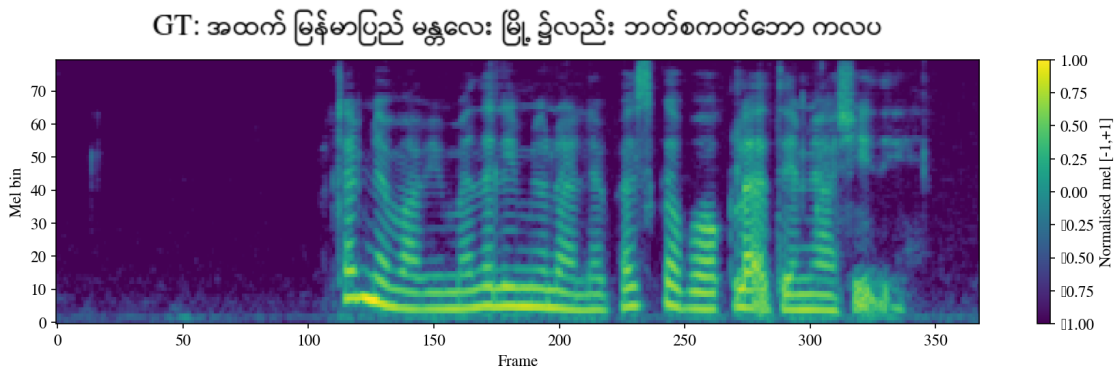
[96]: <IPython.lib.display.Audio object>

```

[97]: from IPython.display import Image
Image(filename='gt_bur_9135_9365873650.png')

```

[97]:



```

[98]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_5932_0137380639

```

```

/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classific
ation/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classific

```

```

ation/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
2026-06-20 19:49:43 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:49:43 [INFO    ] Found bur_5932_0137380639 in train
2026-06-20 19:49:43 [INFO    ] GT text: အရင် တုန်းကတော့ ဒီ ကိစ္စ ကို အားလုံး
လေးငါးခြောက်ယောက်လောက် နဲ့ လုပ်တာ
2026-06-20 19:49:43 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-54.2 dB
shape=(339, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
  plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:49:45 [INFO    ] Mel plot → gt_bur_5932_0137380639.png
2026-06-20 19:49:45 [INFO    ] GT audio → gt_bur_5932_0137380639.wav (3.92s)

real    0m3.812s
user    0m12.285s
sys      0m0.413s

```

```

[99]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_5932_0137380639.wav')

```

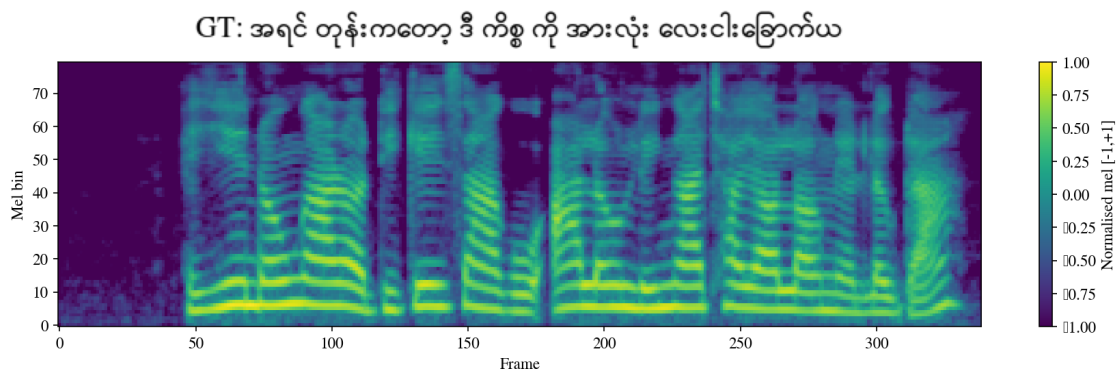
```
[99]: <IPython.lib.display.Audio object>
```

```

[100]: from IPython.display import Image
Image(filename='gt_bur_5932_0137380639.png')

```

```
[100]:
```



```

[101]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_8698_8976831996

```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classific
```

```

ation/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classific
ation/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
2026-06-20 19:50:50 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:50:50 [INFO    ] Found bur_8698_8976831996 in train
2026-06-20 19:50:50 [INFO    ] GT text: မင်း အမေ ကို ကောင်းကောင်း ဆော်သွား
လို့ မင်း ထွက်လာတာ မမေ့နဲ့
2026-06-20 19:50:50 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-63.6 dB
shape=(464, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
  plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:50:52 [INFO    ] Mel plot → gt_bur_8698_8976831996.png
2026-06-20 19:50:52 [INFO    ] GT audio → gt_bur_8698_8976831996.wav (5.38s)

real    0m4.249s
user    0m13.854s
sys      0m0.436s

```

```

[102]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_8698_8976831996.wav')

```

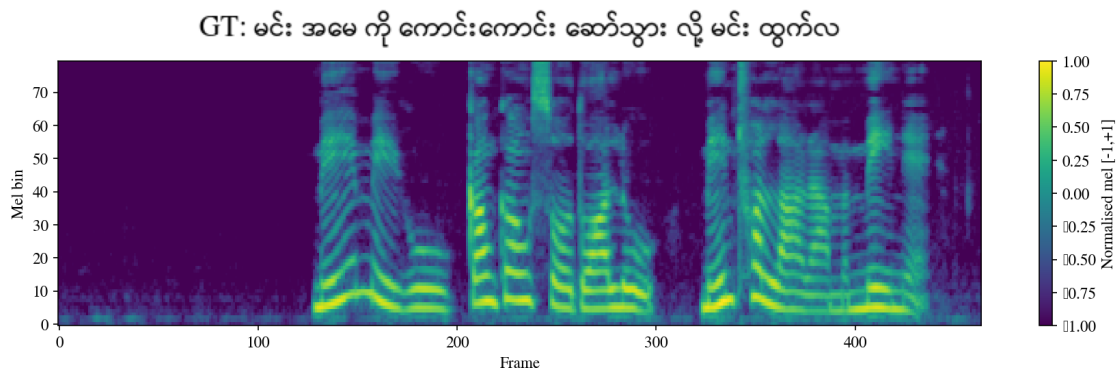
[102]: <IPython.lib.display.Audio object>

```

[103]: from IPython.display import Image
Image(filename='gt_bur_8698_8976831996.png')

```

[103]:




```
[104]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_0366_2701213104
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
2026-06-20 19:52:12 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:52:12 [INFO    ] Found bur_0366_2701213104 in train
2026-06-20 19:52:12 [INFO    ] GT text: ကမ္ဘာအနှံ့မှ သတင်း အချက်အလက်တွေ ကို
သိနိုင်သည်
2026-06-20 19:52:12 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-57.2 dB
shape=(302, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
  plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:52:15 [INFO    ] Mel plot → gt_bur_0366_2701213104.png
2026-06-20 19:52:15 [INFO    ] GT audio → gt_bur_0366_2701213104.wav (3.49s)

real    0m3.702s
user    0m12.074s
sys     0m0.431s
```

```
[105]: import IPython.display as ipd

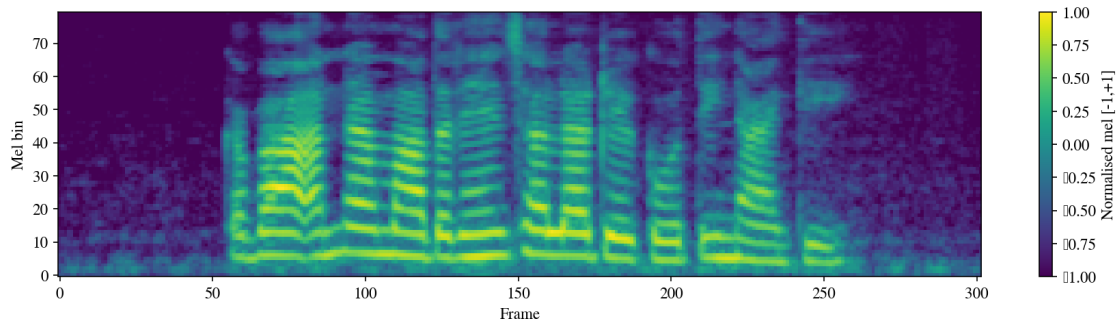
# For a local file
ipd.Audio('./gt_bur_0366_2701213104.wav')
```

```
[105]: <IPython.lib.display.Audio object>
```

```
[106]: from IPython.display import Image
Image(filename='gt_bur_0366_2701213104.png')
```

```
[106]:
```


GT: ကမ္ဘာအနှံ့မှ သတင်း အချက်အလက်တွေ ကို သိနိုင်သည်



```
[107]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_0644_2661360424
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 19:53:21 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:53:21 [INFO    ] Found bur_0644_2661360424 in train
2026-06-20 19:53:21 [INFO    ] GT text: ဆိုနီ ကရော ဘယ်လိုတွေ ချပြမလဲ
လေ့လာကြည့်ရအောင်
2026-06-20 19:53:21 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-54.5 dB
shape=(368, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:53:23 [INFO    ] Mel plot → gt_bur_0644_2661360424.png
2026-06-20 19:53:23 [INFO    ] GT audio → gt_bur_0644_2661360424.wav (4.26s)

real    0m3.920s
user    0m12.669s
sys     0m0.411s
```

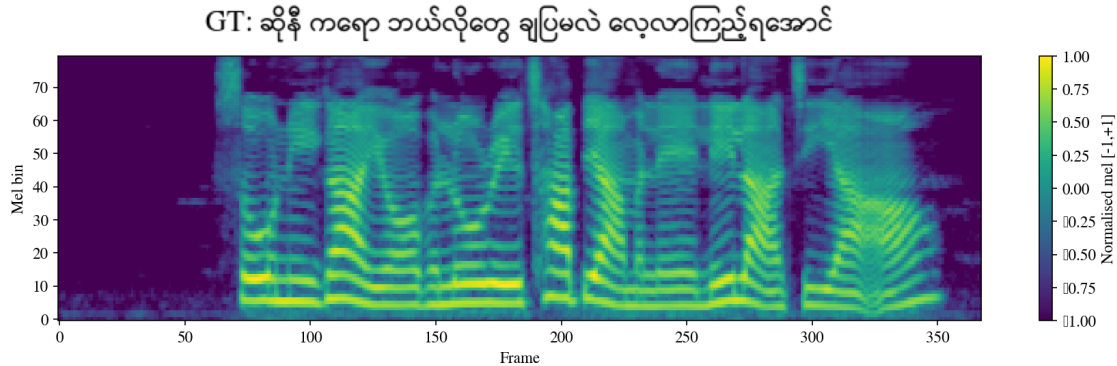
```
[108]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_0644_2661360424.wav')
```

```
[108]: <IPython.lib.display.Audio object>
```

```
[109]: from IPython.display import Image
Image(filename='gt_bur_0644_2661360424.png')
```

```
[109]:
```



```
[110]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_7712_5029179191
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
  warnings.warn(
2026-06-20 19:54:49 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:54:49 [INFO    ] Found bur_7712_5029179191 in train
2026-06-20 19:54:49 [INFO    ] GT text: ကမ္ဘောဇ ဘောလုံး အသင်း က ဥက္ကဋ္ဌ
ဦးခင်မောင်လတ် က အရင်ဆုံး သုံးနှစ် ငှားရမ်းပါတယ်
2026-06-20 19:54:49 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-54.9 dB
shape=(655, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
  plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:54:53 [INFO    ] Mel plot → gt_bur_7712_5029179191.png
2026-06-20 19:54:53 [INFO    ] GT audio → gt_bur_7712_5029179191.wav (7.59s)

real    0m4.907s
user    0m16.280s
sys     0m0.458s
```

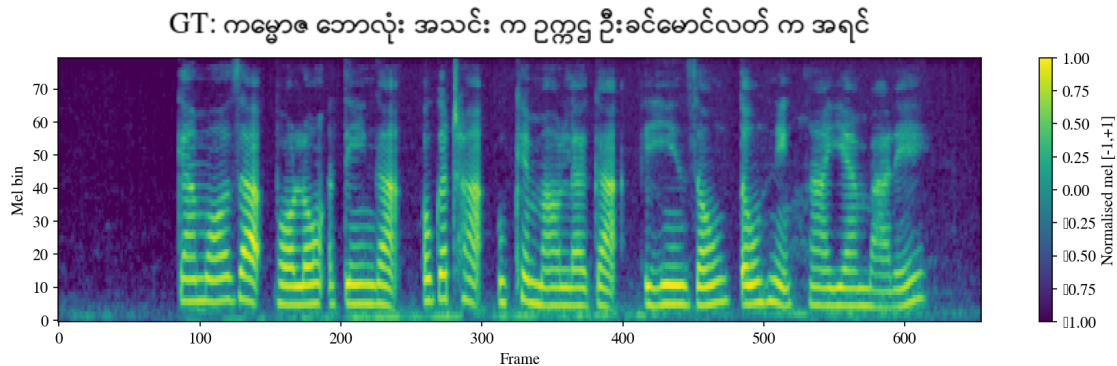
```
[111]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_7712_5029179191.wav')
```

[111]: <IPython.lib.display.Audio object>

```
[112]: from IPython.display import Image
Image(filename='gt_bur_7712_5029179191.png')
```

[112]:



```
[113]: !time python myanmar_tts_transformer.py --stage synth
--output_dir ./tts_output --gt_sample_id bur_5362_4875790905
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 19:56:04 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:56:04 [INFO    ] Found bur_5362_4875790905 in train
2026-06-20 19:56:04 [INFO    ] GT text: ကျွန်တော် ကလည်း ဒိတ်ဒိတ်ကျဲ
အောင်မြင်နေတဲ့ မင်းသား မှ မဟုတ်တာလေ
2026-06-20 19:56:04 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-57.5 dB
shape=(522, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:56:07 [INFO    ] Mel plot → gt_bur_5362_4875790905.png
2026-06-20 19:56:07 [INFO    ] GT audio → gt_bur_5362_4875790905.wav (6.05s)
```

```
real    0m4.469s
user    0m15.053s
sys     0m0.424s
```

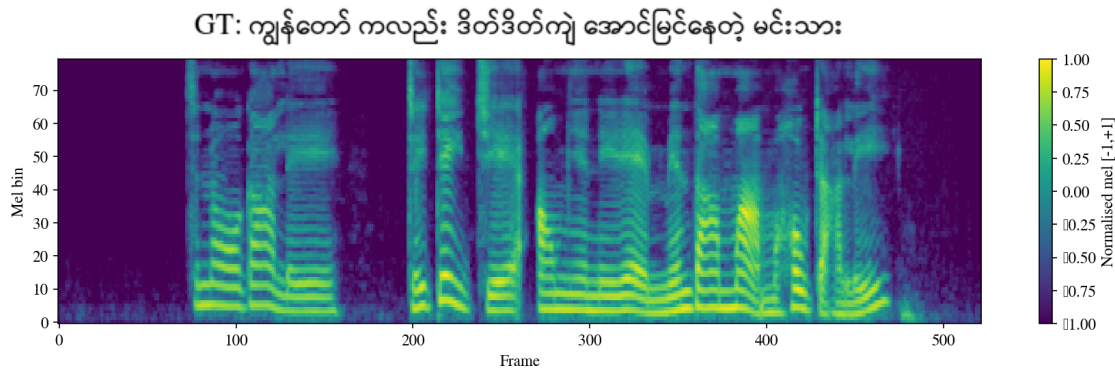
```
[114]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_5362_4875790905.wav')
```

```
[114]: <IPython.lib.display.Audio object>
```

```
[115]: from IPython.display import Image
Image(filename='gt_bur_5362_4875790905.png')
```

```
[115]:
```



```
[116]: !time python myanmar_tts_transformer.py --stage synth \
--output_dir ./tts_output --gt_sample_id bur_3260_2712519099
```

```
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/common.py:13: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
/home/ye/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub/PyTorch/Classification/ConvNets/image_classification/models/efficientnet.py:17: UserWarning:
pytorch_quantization module not found, quantization will not be available
warnings.warn(
2026-06-20 19:57:36 [WARNING ] WaveGlow unavailable (RuntimeError), using
Griffin-Lim.
2026-06-20 19:57:36 [INFO    ] Found bur_3260_2712519099 in train
2026-06-20 19:57:36 [INFO    ] GT text: ကျွန်တော် က ရိက္ခာ ယူခိုင်း တုန်း
ထွက်ပြေးလာတာပါ
2026-06-20 19:57:36 [INFO    ] GT mel: min=-80.0 max=0.0 mean=-56.9 dB
shape=(309, 80)
/mnt/disk1/ye/exp/asr_tts_SLR80/myanmar_tts_transformer.py:812: UserWarning:
```

```

Glyph 8722 (\N{MINUS SIGN}) missing from font(s) Myanmar3.
plt.tight_layout(); plt.savefig(save_path,dpi=150,bbox_inches="tight")
2026-06-20 19:57:38 [INFO ] Mel plot → gt_bur_3260_2712519099.png
2026-06-20 19:57:38 [INFO ] GT audio → gt_bur_3260_2712519099.wav (3.58s)

```

```

real    0m3.730s
user    0m12.185s
sys     0m0.426s

```

```

[117]: import IPython.display as ipd

# For a local file
ipd.Audio('./gt_bur_3260_2712519099.wav')

```

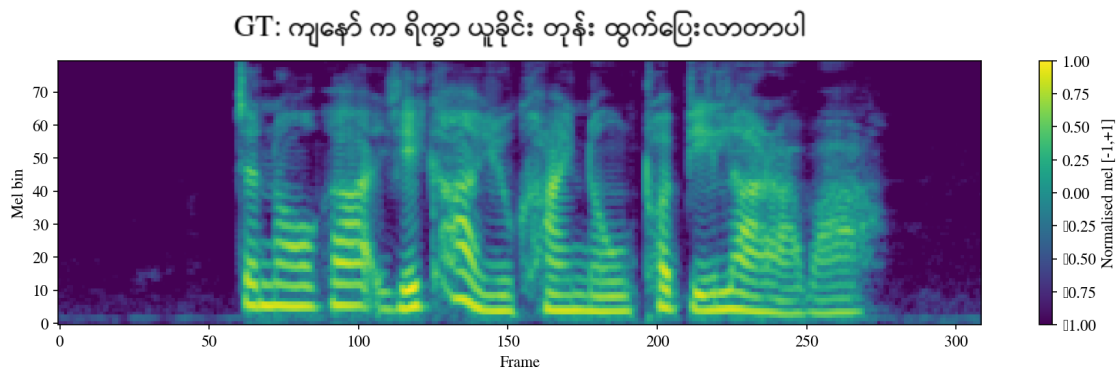
[117]: <IPython.lib.display.Audio object>

```

[118]: from IPython.display import Image
Image(filename='gt_bur_3260_2712519099.png')

```

[118]:



4.7 Summary

- ဒီ TTS Lab (သို့) tutorial အတွက်က Autoregressive Transformer + Guided Attention Loss ကို သုံးခဲ့တယ်။
- ဒီ architecture နဲ့က training loss ကို 0.2408 ထိချနိုင်ခဲ့တာကို တွေ့ရပါလိမ့်မယ်။
- GT resynthesis ကို ကောင်းကောင်းလေး လုပ်ပေးနိုင်ပါတယ်။
- ဒါပေမဲ့ လက်ရှိ approach နဲ့ experiment မှာက TTS model prediction က အဆင်မပြေသေးတာကို တွေ့ခဲ့ကြရပါလိမ့်မယ်။
- အိမ်စာ အနေနဲ့ TTS model ရဲ့ mel spectrogram ရလဒ်ကို တက်အောင် လုပ်ကြည့်ကြပါ။ လိုအပ်ရင် decoder ကိုလည်း ပြောင်းကြည့်ပါ။

[]: